



FORMATION INFORMATIQUE

# Python

Les bases de Python, puis un cran plus loin : fichiers JSON, exceptions, classes.

DanielCraft - Livre debutant

# Sommaire

Clique un chapitre ou un sous-chapitre pour y aller.

## 1. [Chapitre 1 - Salut, c'est quoi Python ?](#)

- [Ce que ce n'est pas](#)
- [Ce que tu vas savoir faire](#)
- [Comment lire ce livre](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : langage vs outil](#)
- [Petite scene DanielCraft](#)
- [Ce que ce livre doit changer chez toi](#)

## 2. [Chapitre 2 - Installer et ecrire Python](#)

- [Ce que ce n'est pas](#)
- [Ou ecrire le code ?](#)
- [Fichier '.py' et dossier propre](#)
- [Lancer](#)
- [La console interactive](#)
- [Virtualenv ? \(aperçu\)](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)

## 3. [Chapitre 3 - Ton premier programme](#)

- [Ce que ce n'est pas](#)
- [print, f-strings, commentaires](#)
- [input : demander quelque chose](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [Exemple complet](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : texte vs nombre](#)

## 4. [Chapitre 4 - Les variables](#)

- [Ce que ce n'est pas](#)
- [Changer une valeur et nommer bien](#)
- [Plusieurs infos, echange, constantes](#)
- [Attention au piege](#)
- [Petite histoire](#)
- [Exemple complet](#)
- [Erreur classique](#)
- [En vrai](#)

- [A toi](#)

## 5. [Chapitre 5 - Les types \(texte, nombre, vrai/faux\)](#)

- [Ce que ce n'est pas](#)
- [Les quatre de base](#)
- [Conversion depuis input](#)
- [type\(\), pièges, None](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : pourquoi Python est "souple"](#)

## 6. [Chapitre 6 - Les conditions \(if\)](#)

- [Ce que ce n'est pas](#)
- [elif, comparaisons, and / or / not](#)
- [Conditions avec texte et ternaire](#)
- [Petite histoire](#)
- [Exemple complet](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : condition vs boucle](#)

## 7. [Chapitre 7 - Les boucles](#)

- [Ce que ce n'est pas](#)
- [for, while, listes](#)
- [break, continue, while True](#)
- [Exemple : table de multiplication](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : range et le piège du dernier nombre](#)

## 8. [Chapitre 8 - Les fonctions](#)

- [Ce que ce n'est pas](#)
- [Paramètres, return, defaults](#)
- [\\*args \(aperçu avancé\)](#)
- [Pourquoi c'est bien](#)
- [Exemple complet](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : nommer, c'est déjà concevoir](#)

## 9. [Chapitre 9 - Les listes](#)

- [Ce que ce n'est pas](#)
- [Modifier, ajouter, parcourir](#)
- [Tranches, tri, comprehension](#)
- [Tuple et set \(aperçus\)](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : index 0, pourquoi ?](#)

#### 10. [Chapitre 10 - Les dictionnaires](#)

- [Ce que ce n'est pas](#)
- [Cle absente, parcours, liste de dicos](#)
- [Compter avec un dico](#)
- [JSON dans la tete](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : dico vs liste, comment choisir](#)

#### 11. [Chapitre 11 - Lire et ecrire un fichier](#)

- [Ce que ce n'est pas](#)
- [Ecrire, lire, modes](#)
- [JSON \(tres utile\)](#)
- [pathlib \(aperçu\)](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : w, a, r en une phrase](#)

#### 12. [Chapitre 12 - Les modules \(import\)](#)

- [Ce que ce n'est pas](#)
- [random, datetime, from import](#)
- [Creer ton module](#)
- [pip \(aperçu\)](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : import math vs from math import sqrt](#)

#### 13. [Chapitre 13 - Mini-projet : quiz terminal](#)

- [Ce que ce n'est pas](#)
- [Version simple](#)
- [Version plus propre](#)

- [Version avec shuffle + JSON](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : donnees vs logique](#)

#### 14. [Chapitre 14 - Ce qu'il faut retenir \(bases\).](#)

- [Ce que ce n'est pas](#)
- [Les idees solides](#)
- [Habitudes qui comptent](#)
- [Erreurs classiques a ne plus digerer](#)
- [Petite histoire](#)
- [Ce que tu peux oublier](#)
- [Suite immediate](#)
- [En vrai](#)
- [A toi](#)
- [Phrase talisman](#)

#### 15. [Chapitre 15 - Les exceptions \(try / except\).](#)

- [Ce que ce n'est pas](#)
- [Plusieurs erreurs, else, finally, raise](#)
- [Ne pas tout avaler + fichier manquant](#)
- [Exemple utile : saisie solide](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : try serre, logique claire](#)

#### 16. [Chapitre 16 - Les classes \(objets\).](#)

- [Ce que ce n'est pas](#)
- [Premiere classe](#)
- [Pourquoi c'est pratique](#)
- [\\_\\_str\\_\\_, heritage, dataclass \(aperçus\).](#)
- [Quand utiliser une classe ?](#)
- [Petite histoire](#)
- [Erreur classique](#)
- [En vrai](#)
- [A toi](#)
- [Zoom : objets et vie reelle](#)

#### 17. [Chapitre 17 - Atelier : lire les erreurs](#)

- [Ce que ce n'est pas](#)
- [Les familles a reconnaitre](#)
- [Methode DanielCraft](#)
- [Exercice 1 - Syntaxe \(8 min\).](#)

- [Exercice 2 - ValueError \(10 min\)](#)
- [Exercice 3 - Fichier \(10 min\)](#)
- [Exercice 4 - Index / cle \(8 min\)](#)
- [Petite histoire](#)
- [Livrable](#)
- [Erreur a eviter](#)
- [En vrai](#)
- [A toi](#)

#### 18. [Chapitre 18 - Atelier : juste prix](#)

- [Ce que ce n'est pas](#)
- [Version de base \(avec try\)](#)
- [Variantes utiles](#)
- [Version objet \(bonus\)](#)
- [Petite histoire](#)
- [Erreur a eviter](#)
- [Livrable](#)
- [En vrai](#)
- [A toi](#)

#### 19. [Chapitre 19 - Atelier : organiser son code](#)

- [Ce que ce n'est pas](#)
- [Version fonctions](#)
- [Pourquoi `if name == "main"` ?](#)
- [Version classe Question](#)
- [Decouper en fichiers \(idee\)](#)
- [Petite histoire](#)
- [Erreur a eviter](#)
- [Livrable](#)
- [En vrai](#)
- [A toi](#)

#### 20. [Quiz final - Teste-toi !](#)

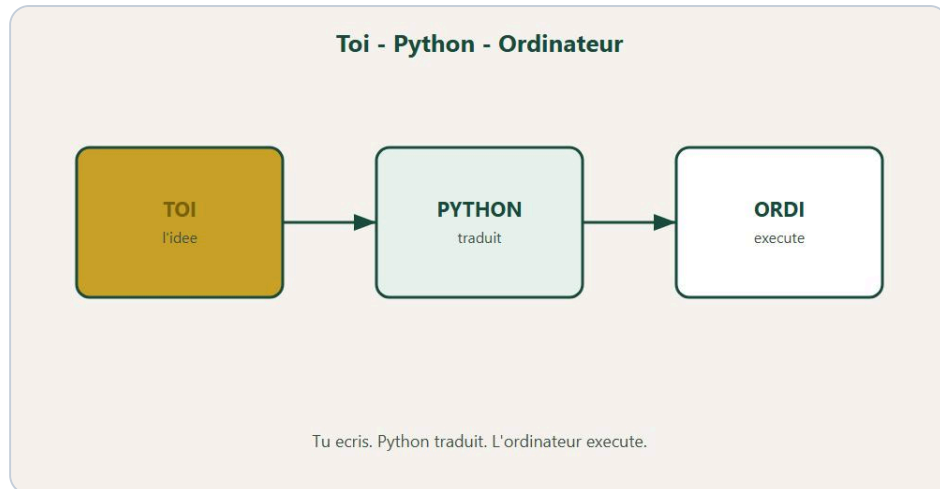
- [Questions](#)
- [Reponses](#)
- [Score](#)
- [Petite histoire](#)
- [En vrai](#)
- [A toi](#)

#### 21. [Bravo.](#)

- [Ce que ce n'est pas](#)
- [Ce que tu sais faire](#)
- [Mission finale](#)
- [Petite histoire](#)
- [La suite \(quand tu voudras\)](#)
- [Engagement de 30 jours](#)

- [Erreur classique apres un livre](#)
- [En vrai](#)
- [A toi](#)
- [Un dernier rappel](#)
- [Petite scene DanielCraft](#)

## Chapitre 1 - Salut, c'est quoi Python ?



Tu écris. Python traduit. L'ordinateur exécute.

Python, ce n'est pas un serment de magie ni un club réservé aux ingénieurs. C'est un **langage de programmation** : une façon d'écrire des instructions que l'ordinateur peut suivre. Tu formules une idée en phrases plus lisibles que dans beaucoup d'autres langages. Python la traduit. La machine exécute. En 2026, quand quelqu'un dit "j'ai fait un petit **script**", il parle souvent de Python : renommer des fichiers, calculer un score, lire un CSV, brancher une API, préparer des données pour un modèle. Le geste reste le même depuis le début : écrire, lancer, lire le résultat, corriger.

Chez DanielCraft, on aime une image nette. Python, c'est un messenger clair entre toi et la machine. Tu restes le chef. Si le message est flou, l'ordinateur se plaint avec un **message d'erreur**. Ce n'est pas une humiliation. C'est un dialogue. Lea, freelance web, s'en sert pour automatiser des exports clients et vérifier des listes. Max, artisan, l'utilise pour un mini calculateur de devis. Sam, enseignant, prépare des quiz dans le terminal pour ses élèves. Trois métiers, même logique : gagner du temps sur le répétitif, garder le cerveau pour le jugement.

### Ce que ce n'est pas

Ce n'est pas "la seule façon de coder". Ce n'est pas non plus un outil qui lit dans tes pensées : si tu n'as pas clarifié le problème, le code sera confus. Ce n'est pas une excuse pour coller un script non relu sur la machine d'un client. Et ce n'est surtout pas un langage "trop simple pour être sérieux". Derrière la syntaxe accueillante, tu peux aller très loin : data, IA, web, automatisation. Ici, on pose des bases solides. Ensuite on touche exceptions et classes, sans devenir un manuel de 800 pages.

Ce n'est pas non plus "tout comprendre avant d'écrire". Beaucoup de débutants attendent d'avoir la théorie complète. Ils n'écrivent jamais la première ligne. Python s'apprend en tapant, en cassant, en relisant l'erreur. Tu n'as pas besoin de devenir chercheur. Tu as besoin de devenir quelqu'un qui ose lancer `python mon_fichier.py` et lire ce qui sort.

Tu as une tâche. Tu la découpes. Tu écris des instructions. Python exécute. Si ça plante, le message d'erreur pointe souvent la ligne. Tu corriges. Tu relances. Le pont, c'est le fichier `.py` : un texte simple, des mots clés, de l'**indentation**. Sans pont clair, tu obtiens du flou. Avec un pont, tu obtiens quelque chose de réutilisable. Plus loin dans le livre : variables, types, conditions, boucles, fonctions, listes, dictionnaires, fichiers, modules, exceptions, classes. Pas pour te faire peur. Pour que tu saches ce que tu manipules.



Lea ouvre VS Code, écrit dix lignes, gagne vingt minutes chaque semaine. Max préfère IDLE au début : moins d'options, moins de panique. Sam projette le terminal devant la classe et montre qu'une erreur n'est pas un drame. Chez DanielCraft, on préfère ce genre d'habitude à la collection de tutos oubliés le lendemain.

#### ♦ ASTUCE

Dis à voix haute une tâche répétée de ton quotidien. Si tu peux la découper en trois étapes claires, tu as déjà un futur script.

## Ce que tu vas savoir faire

Tu vas installer Python et lancer un programme. Tu manipuleras variables, types, conditions et boucles. Tu écriras des fonctions avec paramètres et `return`. Tu joueras avec listes et dictionnaires. Tu liras et écriras des fichiers (texte et JSON). Tu importeras des modules comme `random` ou `math`. Tu gèreras des erreurs avec `try/except`, tu créeras une petite `class`, tu feras un quiz et un jeu dans le terminal. Puis un mini-projet, un recap, trois ateliers, un quiz final, et un bravo.

Niveau débutant solide, avec un cran intermédiaire à la fin. Pas besoin d'avoir déjà codé. Besoin de curiosité et d'honnêteté : tu tapes, tu observes, tu corriges.

## Comment lire ce livre

Lis dans l'ordre au début. Les premiers chapitres posent le sol. Les ateliers font faire. Le quiz vérifie. Retape les exemples à la main. Change une valeur, puis relance. Si tu copies-colles sans réfléchir, ça rentre moins. Si tu tapes, ça rentre. À chaque fin de chapitre, il y a un "À toi". Fais-le. Cinq minutes valent mieux qu'une lecture passive de quarante pages.



Exemple : un `print`, et Salut Python apparaît.

## Petite histoire

Lea devait renommer cinquante captures d'écran pour un client. Avant, elle le faisait à la main, une par une, en pestant. Maintenant, elle écrit un petit script : lire le dossier, construire un nouveau nom, renommer. Elle teste sur trois fichiers. Elle lance sur le reste. Dix minutes, travail net. Python n'a pas "pensé le projet". Il a exécuté une recette claire.

Max, lui, calculait toujours sa marge sur un coin de papier. Il a demandé à Sam une version simple en Python : prix fournitures, temps, marge. Il tape trois nombres, lit le résultat. Ce n'est pas de la triche. C'est de l'aide au calcul - à condition de vérifier que la formule est la sienne, pas une invention. On y revient dans tout le livre : toi tu pilotes, le code accélère.

## Erreur classique

Croire que "je dois tout comprendre avant d'écrire". Ou croire que "le message d'erreur veut dire que je suis nul". Souvent, le problème n'est pas toi. C'est une guillemet oubliée, une indentation, un mauvais dossier dans le terminal. Lis la dernière ligne de l'erreur. Elle parle souvent clairement.

Autre piège : vouloir construire "l'app complète" dès le jour un. Commence par afficher "Salut". Puis une variable. Puis une condition. Monte ensuite. Tu seras prêt.

## En vrai

Ouvre un terminal. Si Python est déjà là, tape `python --version` ou `py --version`. Note le numéro. Si rien ne répond, pas de panique : le chapitre suivant installe. Dis à voix haute une tâche simple que tu aimerais automatiser. Ce livre sert à transformer cette envie en habitude propre.

## A toi

Ecris en trois phrases : (1) une tâche répétée qui te fatigue, (2) ce que tu accepterais qu'un script fasse à ta place, (3) ce que tu ne laisserais jamais tourner sans contrôle humain. Garde ce papier. On y reviendra au mini-projet.

## Zoom : langage vs outil

Beaucoup de gens mélangent Python et "l'IA", ou Python et "un site web". Python est le langage. Les outils (éditeur, bibliothèques, frameworks) s'appuient dessus. Tu peux écrire du Python sans IA. Tu peux utiliser une IA pour t'aider à écrire du Python. Ce n'est pas la même chose. Ici, on apprend le langage. Si tu utilises un assistant pour débloquer une syntaxe, relis toujours ce qu'il propose avant de le coller. Chez DanielCraft, le pilote, c'est toi.

## Petite scène DanielCraft

Lea ouvre son dossier `mes-tests-python`, crée `salut.py`, lance, sourit. Max regarde par-dessus l'épaule, demande "et pour mon devis ?". Sam répond : "d'abord print, ensuite les calculs". Trois usages, une même posture : avancer par petites victoires visibles.

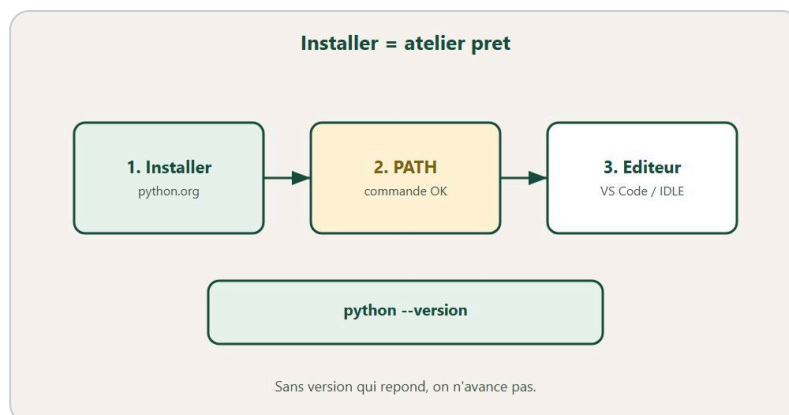
### ★ A RETENIR

Python = un messenger clair. Tu écris la recette, la machine exécute, tu restes responsable.

## Ce que ce livre doit changer chez toi

À la fin, tu ne seras pas architecte logiciel. Tu seras quelqu'un qui lit une erreur sans paniquer, qui découpe un problème, qui range une idée dans une fonction, qui sait ouvrir un fichier et qui ose une petite classe. C'est déjà beaucoup. C'est surtout actionnable demain matin.

## Chapitre 2 - Installer et écrire Python



Installer, PATH, editeur : l'atelier doit répondre.

Avant de coder, il te faut un Python qui répond vraiment. Pas un logo sur un site. Une commande dans le **terminal** qui affiche un numéro de version. Sur Windows, le chemin le plus simple reste souvent le site officiel python.org : tu télécharges la version récente, tu lances l'installateur, et tu coches "**Add Python to PATH**" si tu vois l'option. Ensuite tu ouvres un terminal et tu tapes `python --version`. Si un numéro apparaît, c'est bon. Parfois, sur Windows, c'est `py --version` qui répond mieux. Les deux existent. Garde celui qui marche chez toi et note-le. Lea a perdu une heure à cause du PATH non coché. Max a trouvé `py` plus simple. Sam montre les deux à ses élèves pour éviter la panique.

Chez DanielCraft, on insiste sur ce détail parce que la moitié des "Python ne marche pas" du premier jour sont des histoires d'installation, pas de talent. Tu n'es pas en retard. Tu configures ton atelier.

### Ce que ce n'est pas

Installer Python, ce n'est pas "devenir développeur". Ce n'est pas non plus installer vingt outils d'un coup. Ce n'est pas obligatoire d'ouvrir Docker, WSL, ou une usine à gaz dès le chapitre 2. Un **interpréteur**, un éditeur simple, un dossier de test : ça suffit pour tout ce livre. Plus tard, tu découvriras les environnements virtuels. Pour l'instant, on reste terre à terre.

Ce n'est pas non plus une étape "triviale pour les autres". Beaucoup de gens bloquent ici. Si tu bloques, tu es dans la moyenne. Relis, réessaie, change de terminal, vérifie le PATH. Puis avance.

Ton ordi a une boîte à outils. Python est un outil dans la boîte. Le terminal est la façon de parler à l'outil. L'**éditeur** (VS Code, IDLE...) est l'endroit où tu écris la recette. Le fichier `.py` est la feuille. Quand tu lances `python salut.py`, tu dis : "exécute cette feuille avec cet outil". Si le terminal dit "introuvable", deux causes classiques : mauvais dossier, ou Python pas dans le PATH. Pas de mystère cosmique.

#### ⚠ ATTENTION

Sans PATH coché à l'install, Windows répond souvent "commande introuvable". Ce n'est pas toi. C'est la config. Recoche, reteste.

## Ou ecrire le code ?

Deux options debutant : **VS Code** (gratuit) avec l'extension Python, ou **IDLE** (fourni avec Python), plus basique mais rassurant. Plus tard tu pourras aimer PyCharm ou autre. Pour ce livre, VS Code ou IDLE suffisent largement. Lea travaille dans VS Code parce qu'elle y a deja son HTML. Max prefere IDLE au debut : moins de panneaux. Sam alterne selon le cours. Choisis, note ton choix, n'en change pas toutes les cinq minutes.

## Fichier `.py` et dossier propre

Cree un dossier `mes-tests-python`. Dedans, cree un fichier `salut.py`. C'est ton programme. Astuce : un dossier par projet. Evite de tout mettre sur le Bureau en vrac. Quand tu auras dix fichiers, tu remercieras cette habitude. Chez DanielCraft, on traite le rangement comme une competence, pas comme du menage optionnel.

## Lancer

Dans le dossier du fichier :

```
python salut.py
```

Ou :

```
py salut.py
```

Si le terminal dit "introuvable", c'est souvent le mauvais dossier (tu n'es pas au bon endroit), ou le PATH pas coche a l'install. Utilise `cd` pour te deplacer vers le bon dossier, puis relance. Lea tape souvent `pwd` ou `cd` avant de se plaindre. Bon reflexe.

## La console interactive

```
python
```

Tu peux tester des lignes une par une. Utile pour un calcul vite fait.

```
>>> 2 + 2
4
>>> exit()
```

`exit()` pour quitter. Ou Ctrl+Z puis Entree sur Windows. Sam utilise la console devant la classe pour montrer qu'une idee se teste sans creer un fichier. Max s'en sert pour verifier `19.99 * 1.2` avant de le mettre dans un script.

## Virtualenv ? (aperçu)

Plus tard, les projets serieux utilisent un "**environnement virtuel**". Idee : isoler les bibliotheques d'un projet pour ne pas melanger les versions.

```
python -m venv .venv
```

Puis on active, puis `pip install ...`. Pour l'instant, tu n'en as pas besoin. Mais le mot existe. Tu le reverras. Mieux vaut l'avoir entendu une fois que de le decouvrir en panique le jour J.

#### ♦ ASTUCE

Note dans `notes.md` : version Python + commande qui marche ( `python` ou `py` ) + editeur choisi. Ce papier te sauvera dans trois semaines.

### Petite histoire

Lea a installe Python trois fois avant de cocher le PATH. La quatrieme, `python --version` a repondu. Elle a cree `mes-tests-python`, lance un fichier vide, vu "rien", sourit quand meme : l'atelier etait pret. Max a demande a Sam de verifier avec lui. Sam a regarde le message d'erreur, a dit "mauvais dossier", a montre `cd`. Trois minutes. Chez DanielCraft, on celebre ces petites victoires : elles annoncent les grandes.

### Erreur classique

Tu lances `python salut.py` depuis le mauvais dossier. Le terminal dit qu'il ne trouve pas le fichier. Solution : `cd` vers le bon dossier, puis relance. Autre classique : installer Python sans PATH, puis croire que "Python est casse". Reinstalle en cochant l'option, ou cherche comment ajouter Python au PATH sur ta version de Windows. Puis reteste.

### En vrai

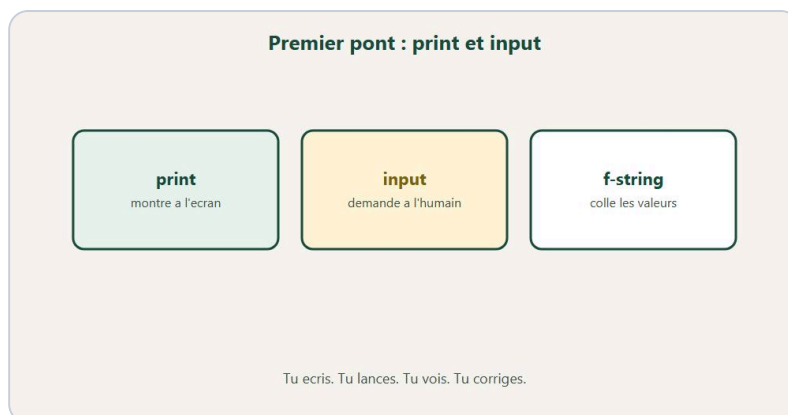
Ouvre le terminal. Tape la commande de version. Note le resultat sur papier ou dans un fichier `notes.md`. Si ca marche, tu es pret pour ecrire. Si ca ne marche pas, reste ici jusqu'a ce que ca marche. Le chapitre 3 suppose un Python qui repond.

### A toi

1. Verifie `python --version` (ou `py --version`).
2. Cree le dossier `mes-tests-python` et le fichier `salut.py`.
3. Cree aussi `notes.md` : version Python + outil choisi (VS Code / IDLE).

On ecrira dans `salut.py` au chapitre suivant.

## Chapitre 3 - Ton premier programme



print montre. input demande. f-string colle.

Ouvre `salut.py` et écris une seule ligne claire :

```
print("Salut Python")
```

Lance ensuite :

```
python salut.py
```

Si tu vois `Salut Python`, bravo. Ce n'est pas un gadget. C'est le premier pont solide entre ton texte et l'écran. Chez DanielCraft, on traite ce moment comme une cérémonie discrète : tu as donné un ordre, la machine a obéi. Lea sourit encore la première fois. Max a dit "c'est tout ?". Sam a répondu : "c'est le début de tout".

A partir d'ici, chaque chapitre ajoute une brique. Tu n'as pas besoin de tout inventer. Tu as besoin de retaper, changer une valeur, relancer, observer.

### Ce que ce n'est pas

Un premier programme, ce n'est pas une application complète. Ce n'est pas non plus "trop simple pour compter". Beaucoup de gens veulent un jeu, un site, un bot dès le jour un. Ils se noient. Ici, on commence par afficher. Ensuite on demande. Ensuite on décide. Ensuite on répète. La progression est volontairement lente - et efficace.

Ce n'est pas non plus "copier-coller une fois et passer". Si tu ne retapes pas, ton doigt n'apprend pas, et ton œil ne voit pas les petites fautes. Retape. Change la phrase. Relance.

**print**(...) = "montre ça à l'écran". Le texte va entre guillemets. Les nombres peuvent vivre sans guillemets. Tu peux mélanger plusieurs choses séparées par des virgules : Python ajoute un espace entre. Plus loin, les **f-strings** collent des variables dans une phrase lisible. **input** demande quelque chose à l'humain. Attention : `input` renvoie toujours du texte, même si tu as tapé `12`.

#### ★ A RETENIR

`print` montre. `input` demande. Les f-strings collent des valeurs dans une phrase claire.

## print, f-strings, commentaires

```
print("Coucou")
print(2 + 2)
print("2 + 2 =", 2 + 2)
```

Les f-strings (pratique quotidienne) :

```
prenom = "Leo"
age = 12
print(f"Je m'appelle {prenom} et j'ai {age} ans")
```

Le `f` devant la chaîne laisse coller des variables avec `{...}`. Plus lisible que coller avec `+` partout. Lea les utilise dans presque tous ses scripts clients. Max les a adoptés pour ses messages de devis.

### Commentaires :

```
# Ceci est une note pour toi. Python l'ignore.
print("Je compte") # commentaire en fin de ligne
```

Ecris des commentaires utiles, pas "ici j'affiche". Plutôt : pourquoi tu fais un truc pas évident. Sam interdit les commentaires qui répètent le code mot pour mot.

## input : demander quelque chose

```
prenom = input("Ton prenom ? ")
print("Salut", prenom)
print(f"Enchante, {prenom} !")
```

`input` attend que tu écrives puis Entree. Souvent, on nettoie :

```
reponse = input("Ville ? ").strip()
```

`strip()` enlève les espaces avant/après. Utile. Les gens tapent souvent un espace sans le voir. Max a passé dix minutes à chercher pourquoi `"Lyon "` n'égalait pas `"Lyon"`. `strip()` a réglé le mystère.

### ♦ ASTUCE

Retape chaque exemple à la main. Changer une seule phrase et relancer vaut mieux que coller dix tutos.

## Petite histoire

Lea a écrit un mini script de présentation pour un atelier : prenom, hobby, ville. Elle l'a lancé devant un client pour montrer "voir, c'est lisible". Le client a souri. Pas parce que c'était génial techniquement. Parce que c'était compréhensible. Chez DanielCraft, on aime ce genre de preuve : le code sert un humain, pas l'inverse.

Sam fait retaper l'exemple à chaque élève. Ceux qui collent bloquent plus tard. Ceux qui tapent avancent plus vite au chapitre variables. La leçon n'est pas morale. Elle est musculaire.

## Erreur classique

Oublier les guillemets :

```
print(Salut) # erreur : Salut n'existe pas comme variable
```

Le message d'erreur pointe souvent la ligne. Lis-la. Autre classique : confondre le fichier ouvert dans l'editeur et le fichier lance dans le terminal. Tu modifies `salut.py`, mais tu lances `test.py`. Relis le nom. Respire. Relance.

## Exemple complet

```
# mini presentation
prenom = input("Prenom ? ").strip()
hobby = input("Hobby ? ").strip()
print(f"Salut {prenom}. Cool pour {hobby}.")
print("Fin du programme.")
```

Retape-le. Change les phrases. Ajoute une troisieme question si tu veux. Puis seulement, passe a "A toi".

### En vrai

Retape l'exemple sans copier-coller. Change les phrases. Relance. Regarde. Si une erreur apparait, ne ferme pas le terminal en panique : lis la derniere ligne, corrige, relance. C'est le vrai sport du debutant.

### A toi

Demande le prenom et l'age (age en texte pour l'instant). Affiche : `Bonjour, <prenom> ! Tu as <age> ans.` avec une f-string. Bonus : ajoute une 3e question (ville ou animal prefere) et affiche une mini bio en 2 lignes.

## Zoom : texte vs nombre

Pour l'instant, l'age reste du texte. `"12" + "1"` donnerait `"121"`, pas `13`. Au chapitre types, tu convertiras avec `int(...)`. Ici, le but est plus simple : parler avec `print` et `input` sans te noyer. Une competence a la fois. Chez DanielCraft, on appelle ca "ne pas empiler les mysteres".



## Chapitre 4 - Les variables



Une variable, c'est une boîte avec un nom.

Une **variable**, ce n'est pas un concept abstrait de matheux. C'est une boîte avec un nom. Tu ranges une valeur. Tu la reutilises plus tard. Tu peux changer le contenu. Le nom reste. En Python, tu n'as pas besoin d'annoncer "c'est un nombre" ou "c'est du texte" à l'avance : Python regarde la valeur et comprend. Lea range un `prix_ht`. Max range un `nombre_de_vies` pour un mini jeu. Sam range un `score_eleve`. Trois boîtes, même idée.

Chez DanielCraft, on insiste sur les noms clairs dès le début. Pas parce qu'on aime le style. Parce qu'un mauvais nom te coûte dix minutes plus tard, et parfois une heure en équipe.

```
age = 12
prenom = "Leo"
score = 0
```

### Ce que ce n'est pas

Une variable, ce n'est pas une "formule magique". Ce n'est pas non plus une constante éternelle, même si tu peux adopter une convention MAJUSCULES pour signaler "je ne veux pas changer ça". Ce n'est pas une excuse pour écrire `x1`, `a`, `tmp2` partout. Et ce n'est surtout pas un tiroir où tu ranges n'importe quoi sans regarder le type : tu peux écrire `score = "dix"` après `score = 10`, Python l'accepte, mais ton futur toi ne te remerciera pas.

Imagine des boîtes sur une étagère. L'étiquette dit `score`. Dedans, tu mets `0`. Plus tard, tu remplaces par `10`. Quand tu écris `score + 1`, tu regardes dans la boîte, tu calcules, souvent tu remets le résultat dans la boîte avec `score = score + 1` ou le raccourci `score += 1`. Si tu confonds l'étiquette et le contenu, tu te perds. Si tu gardes des étiquettes parlantes, tu te retrouves.

#### ♦ ASTUCE

Avant d'écrire le code, nomme tes boîtes à voix haute : `prenom`, `points`, `max`. Si le nom sonne flou, change-le.

## Changer une valeur et nommer bien

```
score = 0
score = score + 1
score += 5 # raccourci : score = score + 5
print(score)
```

Oui :

```
nombre_de_vies = 3
message_accueil = "Salut"
```

Non (ou alors on se perd) :

```
x1 = 3
a = "Salut"
```

En Python, on utilise souvent le **snake\_case** : mots separes par `_`. C'est la convention. Suis-la. Ton futur toi te remerciera. Lea refuse les variables d'une lettre dans ses scripts clients, sauf boucles tres courtes. Sam enleve des points si le nom ne dit rien.

## Plusieurs infos, echange, constantes

```
a = 3
b = 5
total = a + b
print(total)
```

Tu peux aussi echanger :

```
x = 1
y = 2
x, y = y, x
print(x, y) # 2 1
```

Python n'a pas de vrai `const` comme certains langages. Par convention, on ecrit en **MAJUSCULES** ce qu'on ne veut pas changer :

```
MAX_ESSAIS = 5
```

Ce n'est pas bloque techniquement. C'est un signal pour les humains. Max met `TAUX_TVA = 0.20` en haut de son calculateur. Quand le taux change, il sait ou regarder.

## Attention au piege

```
# piege : ecraser sans faire gaffe
score = 10
score = "dix" # autorise, mais souvent une mauvaise idee
```

Une variable peut changer de type. Ca ne veut pas dire que c'est clair. Si tu melanges nombres et textes dans la meme boite selon l'humeur, les bugs arrivent en silence puis explosent plus loin.

#### ⚠ ATTENTION

Une variable contient une valeur, pas une formule vivante. Si tu changes `a` apres avoir calcule `total`, `total` ne se met pas a jour tout seul.

## Petite histoire

Lea a herite d'un script ou tout s'appelait `data`, `data2`, `tmp`. Elle a passe une soiree a renommer. Le script n'etait pas "plus intelligent" apres. Il etait lisible. Le client a pu le comprendre. Chez DanielCraft, on appelle ca de la gentillesse envers le prochain lecteur - souvent toi dans trois semaines.

Max a cree un mini profil joueur : pseudo, niveau, xp. Il a change le seuil, relance, vu le "niveau up". Il a compris les variables mieux qu'avec une definition scolaire. Sam projette le meme exemple et demande aux eleves de renommer avant d'ajouter une regle.

## Exemple complet

```
pseudo = "PixelFox"
niveau = 1
xp = 0

xp += 50
print(f"{pseudo} a {xp} xp")

if xp >= 50:
    niveau += 1
    xp -= 50
    print(f"Niveau up ! Niveau {niveau}")

print(f"Etat : niveau {niveau}, xp {xp}")
```

Meme si `if` arrive plus en detail au chapitre suivant, tu peux deja lire l'idee : la boite change, une decision suit.

## Erreur classique

Ecraser une variable importante par accident, ou reutiliser le meme nom pour deux idees differentes. Autre classique : croire que le nom de la variable "calcule tout seul". Non. `total` ne se met pas a jour si tu changes `a` apres coup, sauf si tu recalculs. La boite contient une valeur, pas une formule vivante (sauf si tu reconstruis volontairement cette logique).

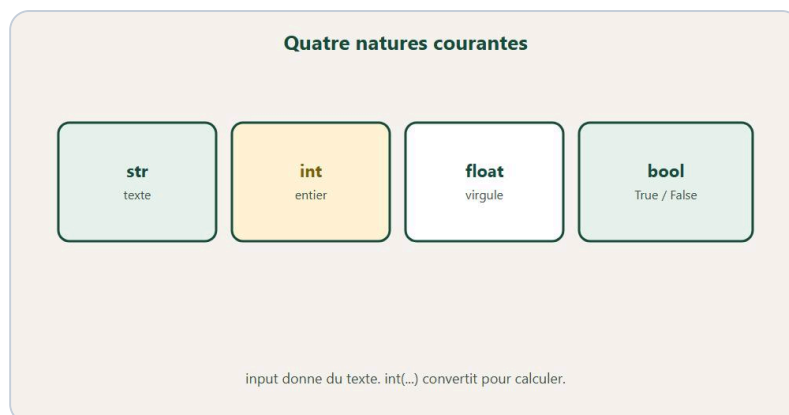
## En vrai

Retape l'exemple joueur. Change le pseudo et le seuil d'xp. Relance. Puis invente un mini profil (jeu, sport, ecole) avec 3 variables minimum et une phrase complete affichee.

## A toi

Cree une variable `prenom` et une variable `points` qui commence a 0. Ajoute 10 points, puis affiche le tout avec `print` ou une f-string. Bonus : ajoute `MAX_POINTS = 100` et affiche combien il reste avant le max.

## Chapitre 5 - Les types (texte, nombre, vrai/faux)



str, int, float, bool : la nature de la boîte.

Les **types**, c'est la nature de ce que tu ranges dans la boîte. Texte, nombre entier, nombre à virgule, vrai/faux : ce n'est pas du jargon pour faire joli. C'est la raison pour laquelle `"3" + "1"` donne `"31"` et `3 + 1` donne `4`. Si tu ignores le type, tu te bats contre des fantômes. Si tu le regardes, les erreurs deviennent lisibles.

Chez DanielCraft, on enseigne les types tôt parce que `input` renvoie toujours du texte. Lea a déjà envoyé un script client qui collait des chiffres au lieu de les additionner. Max a cru que son devis "additionnait" alors qu'il concaténait. Sam commence chaque atelier par `type(...)` au tableau. Une habitude, pas un détail. Tu gagnes des heures en posant cette question tôt : "qu'est-ce que j'ai vraiment dans la boîte ?"

### Ce que ce n'est pas

Connaitre les types, ce n'est pas mémoriser un catalogue académique. Ce n'est pas non plus "annoter tous tes programmes comme en Java" dès le jour un. Ce n'est pas une excuse pour avoir peur de convertir. Et ce n'est surtout pas optionnel : sans conversion, tes calculs à partir de `input` mentent gentiment. Un mensonge poli, c'est encore un mensonge - surtout sur un devis.

Quatre boîtes courantes. **str** : une chaîne de caractères, entre guillemets. **int** : un entier. **float** : un nombre à virgule. **bool** : `True` ou `False`, avec majuscule. Plus tard, listes et dictionnaires. Pour l'instant, maîtrise ces quatre. Quand quelque chose cloche, demande-toi : "qu'est-ce que j'ai vraiment dans la boîte ?" Puis vérifie avec `type()`. Lea le fait encore. Max aussi. Sam l'exige avant toute aide.

#### ♦ ASTUCE

`input("Age ? ")` donne `"12"` (texte). `int(...)` transforme en `12` (nombre). Sans ça, `"12" + "1"` devient `"121"`.

### Les quatre de base

```
phrase = "Bonjour"
ville = 'Lyon' # simples ou doubles, au choix
vies = 3
prix = 19.99
est_connecte = True
a_fini = False
```

Attention : `True` / `False` avec majuscule. Pas `true` comme en JavaScript. Lea a perdu cinq minutes la-dessus en passant d'un langage a l'autre. Normal. Note-le. Sam met un post-it "True avec T" sur le coin de l'ecran la premiere semaine. Ca marche.

## Conversion depuis input

`input` donne du texte. Pour calculer, convertis :

```
age_texte = input("Age ? ")
age = int(age_texte)
print("Dans 10 ans :", age + 10)
```

Versions courtes :

```
n = int(input("Nombre ? "))
prix = float(input("Prix ? "))
```

Si la personne ecrit "douze", `int(...)` plante. Normal. Au chapitre exceptions, on apprendra a rattraper ca. Ici, tu acceptes que le programme exige un vrai nombre. Max teste toujours avec `abc` pour voir l'erreur. Bon reflexe de pilote. Tu ne cherches pas a tout proteger encore. Tu cherches a comprendre ce qui casse.

## `type()`, pieges, None

```
print(type(3))      # int
print(type("3"))    # str
print(type(3.0))    # float
print(type(True))   # bool
```

Pieges classiques :

```
print("3" + "1")    # "31" (colle du texte)
print(3 + 1)         # 4
# print("3" + 1)     # TypeError
```

Et :

```
print(bool(""))      # False (texte vide)
print(bool("0"))     # True (texte non vide)
print(bool(0))        # False
```

**None** veut dire "pas de valeur" volontaire :

```
valeur = None
```

Tu le croiseras avec les fonctions qui ne renvoient rien. Sam dit : "`None`", c'est le silence explicite". Utile comme image. Lea l'utilise parfois comme "pas encore calcule". Max prefere un message clair quand c'est une erreur - on verra ca avec les exceptions.

### ⚠ ATTENTION

`bool("False")` vaut `True` : la chaine n'est pas vide. Convertis d'abord, compare ensuite.

## Petite histoire

Max a demande un prix, ajoute "20%" a la main dans sa tete, puis a voulu le coder. Premier essai : `prix + "20%"`. Plantage. Deuxieme essai : il convertit en `float`, calcule `prix * 1.20`, affiche. Le devis devient honnete. Lea, elle, utilise `float` pour des montants et garde les arrondis sous controle avec `round(...)` quand elle affiche. Chez DanielCraft, on prefere un calcul clair a une magie opaque.

Autre scene : Sam ecrit `"12" == 12` au tableau. Les eleves votent. Beaucoup disent vrai. Python dit `False`. Silence. Puis `type` sur chaque cote. Le clic mental arrive. C'est ca qu'on veut : voir avant de theoriser.

## Erreur classique

Additionner sans convertir. Ou croire que `bool("False")` est `False` : non, c'est `True`, parce que la chaine n'est pas vide. Autre piege : comparer un nombre et un texte (`12 == "12"` est `False`). Convertis d'abord, compare ensuite. Et encore : croire que `float("19,99")` marche avec une virgule francaise - souvent non. Remplace, ou normalise, puis convertis.

## En vrai

Ouvre la console interactive. Teste `type()` sur 4 valeurs differentes. Puis ecris un mini script TVA : demande un prix en texte, convertis en `float`, ajoute 20%, affiche le TTC. Verifie a la main avec une calculatrice. Si les deux resultats collent, tu pilotes. Si non, regarde le type avant de paniquer.

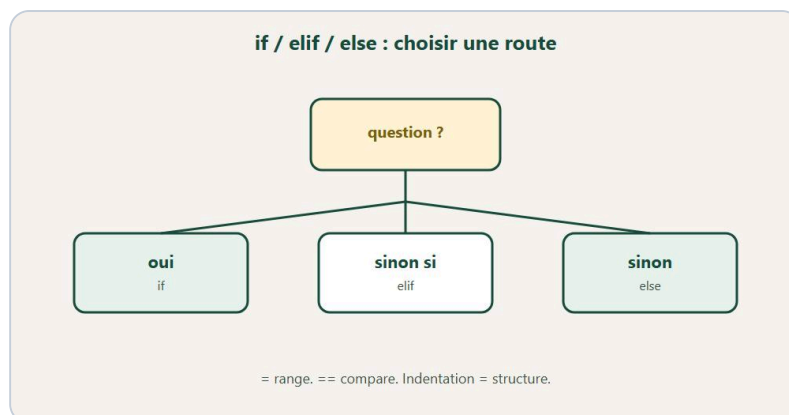
## A toi

Demande deux nombres (via `input + int`). Affiche leur somme, leur difference, leur produit. Bonus : affiche aussi le type de chaque resultat avec `type(...)`. Super bonus : demande un prix en `float` et affiche le TTC arrondi a deux decimales avec `round(...)`.

## Zoom : pourquoi Python est "souple"

Python ne t'oblige pas a declarer le type a l'avance. Ca va vite. Ca peut aussi cacher des erreurs. La souplesse n'est pas une permission d'etre flou. C'est une invitation a verifier. Quand tu doutes, `print(valeur, type(valeur))` est ton ami. Lea le laisse parfois en commentaire de debug cinq minutes, puis l'enleve. Sam oblige les eleves a le faire avant de demander de l'aide. Chez DanielCraft, on appelle ca "voir avant de theoriser". Tu garderas ce geste toute ta vie de code, meme avec des outils plus riches.

## Chapitre 6 - Les conditions (if)



if / elif / else : le programme choisit une route.

Les **conditions**, c'est le moment où ton programme cesse d'être une suite linéaire et commence à choisir. "Si ... alors ... sinon ...". Sans conditions, tu affiches toujours la même chose. Avec des conditions, tu adaptes : majeur ou mineur, mot de passe OK ou refuse, score suffisant ou pas. Lea s'en sert pour valider un formulaire avant d'envoyer un mail. Max pour savoir si sa marge de devis est correcte. Sam pour noter automatiquement un quiz simple. Trois métiers, même muscle : poser une question claire, puis prendre une route.

Chez DanielCraft, on insiste sur l'**indentation** dès le premier **if**. En Python, le décalage n'est pas décoratif. C'est la structure. Quatre espaces. Sans ça, Python refuse. Ce n'est pas une lubie : c'est la façon dont le langage lit les blocs. Si tu viens d'un autre langage avec des accolades, ce sera le premier choc. Ensuite, tu ne pourras plus t'en passer : le code se lit comme un plan.

```

age = 15

if age >= 18:
    print("Majeur")
else:
    print("Mineur")
  
```

### Ce que ce n'est pas

Une condition, ce n'est pas "de la logique avancée réservée aux experts". Ce n'est pas non plus une excuse pour imbriquer dix **if** illisibles dans une seule fonction. Ce n'est pas la même chose qu'une boucle : ici tu décides une fois (ou quelques fois), tu ne répètes pas automatiquement. Et ce n'est surtout pas **=** pour comparer : **=** assigne, **==** compare. Ce piège revient chez presque tout le monde, Lea et Max inclus. Sam le note en rouge au tableau dès la première semaine.

Ce n'est pas non plus "tout tester d'un coup". Une condition claire vaut mieux qu'un monstre de **and** / **or** que tu ne comprends plus demain matin. Découpe. Nomme. Relis à voix haute.

Tu arrives à un carrefour. Une question claire. Si oui, tu prends une route. Sinon, une autre. Parfois il y a plusieurs panneaux : **elif** = "sinon si". Tu peux en chaîner plusieurs. À la fin, un **else** rattrape le reste. Plus le carrefour est clair, moins tu te perds. Si ta question est floue (**if score:** sans savoir ce que tu testes vraiment), le comportement devient étrange. Lea lit ses conditions à voix haute avant de coder. Si la phrase orale est confuse, le code le sera aussi.

Chez DanielCraft, on aime cette habitude : d'abord la phrase humaine, ensuite les symboles. "Si j'ai un ticket ou si je suis VIP, j'entre". Puis seulement `if ticket or vip:`.

#### ⚠ ATTENTION

`=` range une valeur. `==` compare. Melanger les deux est le piege numero un des conditions.

### elif, comparaisons, and / or / not

```
note = 14

if note >= 16:
    print("Excellent")
elif note >= 10:
    print("Valide")
else:
    print("On revise")
```

Comparaisons utiles : `==`, `!=`, `>`, `<`, `>=`, `<=`. Tu les croiseras partout. Apprends-les comme le code de la route : pas pour briller, pour ne pas te planter au premier croisement.

```
ticket = True
vip = False

if ticket or vip:
    print("Entre")

if ticket and vip:
    print("Acces premium")

if not vip:
    print("Pas VIP")
```

**and** exige les deux. **or** accepte l'un ou l'autre. **not** inverse. Max a mis longtemps a confondre **and** et **or** sur ses devis : "si marge OK et client OK" n'est pas "si marge OK ou client OK". Une seule lettre change le metier.

### Conditions avec texte et ternaire

```
mdp = input("Mot de passe ? ").strip()
if mdp == "python123":
    print("OK")
else:
    print("Refuse")
```

Souvent on compare en minuscules, parce que les humains tapent comme ils veulent :

```
if mdp.lower() == "python123":
    print("OK")
```

Operateur ternaire (aperçu) :



```
age = 20
statut = "majeur" if age >= 18 else "mineur"
print(statut)
```

Pratique pour une petite decision. Pas obligatoire. Sam dit : "si tu as besoin d'un commentaire pour comprendre ton ternaire, écris un `if` normal". Chez DanielCraft, la lisibilité gagne toujours contre la ruse d'une ligne.

## Petite histoire

Max a codé un "si `temperature < 0` -> froid". Premier essai, il a mis `=` au lieu de `==`. Python a protesté ou a fait autre chose selon le cas. Deuxième essai, correctement comparé. Il a testé -2, 10, 25. Trois sorties différentes. Il a compris mieux qu'avec un cours abstrait. Lea, elle, valide des seuils de budget avant d'envoyer un mail automatique. Sam a fait écrire à ses élèves trois branches pour une note, puis a demandé : "et si la note vaut exactement 10 ?" Ceux qui avaient mis `>` au lieu de `>=` ont vu le piège. Une histoire, une leçon durable.

### ♦ ASTUCE

Teste toujours les frontières : `note = 10`, `temperature = 0`. C'est là que les bugs aiment se cacher.

## Exemple complet

```
score = int(input("Score ? "))
vies = int(input("Vies ? "))

if score >= 100 and vies > 0:
    print("Niveau suivant")
elif vies == 0:
    print("Game over")
else:
    print("Continue")
```

Change les seuils. Relance avec 9, 10, 16 pour une note. Observe. C'est comme ça que ça rentre. Ne te contente pas d'un seul test "heureux". Teste aussi le bord : égalité, zéro, valeur absurde.

## Erreur classique

Oublier l'indentation. Utiliser `=` au lieu de `==`. Écrire des conditions impossibles ( `if age > 18 and age < 10` ). Ou trop faire confiance à une comparaison de texte sans `.strip()` / `.lower()` . Autre piège : enchaîner des `if` séparés quand tu voulais `elif` - plusieurs blocs peuvent alors s'exécuter sans que tu le veuilles. Lea a déjà envoyé deux messages clients le même jour à cause de ça. Depuis, elle relit ses branches à voix haute.

## En vrai

Programme "météo" : si `temperature < 0` -> "froid", `< 20` -> "doux", sinon "chaud". Teste au moins trois valeurs. Puis refais l'exemple note avec tes propres seuils. Note sur papier ce qui se passe à la frontière (0, 20). Les frontières, c'est là que les bugs aiment se cacher.

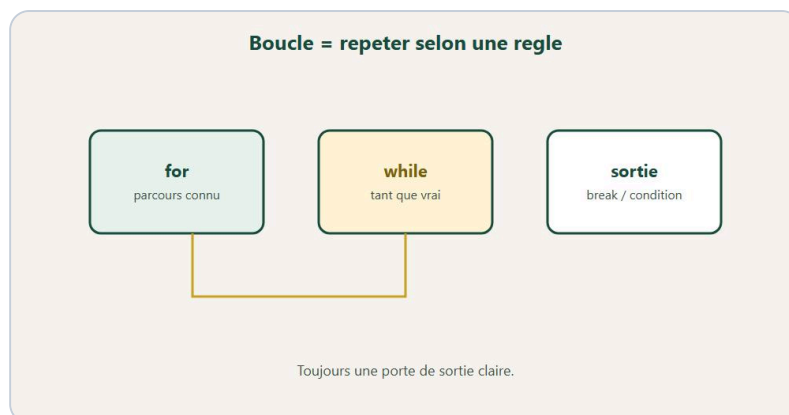
## A toi

Demande un mot de passe. Si c'est `"python123"`, affiche "OK". Sinon "Refuse". Bonus : accepte aussi avec majuscules mixees via `.lower()`. Puis ecris une version avec `and` : mot de passe OK et age  $\geq$  12. Garde ce petit script. On le reutilisera quand on parlera d'erreurs de conversion avec `int(input(...))`.

## Zoom : condition vs boucle

Beaucoup de debutants melangent les deux. La condition decide. La boucle repete. Tu peux mettre une condition dans une boucle ("tant que le mot de passe est faux"), et une boucle dans une branche ("si VIP, afficher chaque privilege"). Mais ce ne sont pas les memes outils. Si tu te surprends a copier dix fois le meme `if`, tu as probablement besoin d'une boucle. Si tu te surprends a tourner sans fin pour "decider une fois", tu as probablement besoin d'un `if` plus clair.

## Chapitre 7 - Les boucles



for ou while : repeter avec une sortie claire.

Une **boucle**, c'est repeter sans tout retaper. Afficher une table de 1 a 10. Demander un mot de passe jusqu'a trois essais. Parcourir une liste de fruits. Sans boucle, tu copies des lignes. Avec une boucle, tu decris la regle une fois. Lea boucle sur des fichiers a renommer. Max boucle sur des lignes de devis. Sam boucle sur des questions de quiz. Meme muscle, contextes differents. En 2026, des que tu automatises un peu, tu touches une boucle - souvent sans meme le remarquer dans un script plus gros.

Chez DanielCraft, on apprend deux familles tot : **for** quand tu sais combien de tours (ou tu parcours une collection), **while** quand tu continues tant qu'une condition reste vraie. Les deux sont utiles. Les deux peuvent devenir dangereuses si tu oublies d'avancer. Une boucle sans sortie, c'est un moteur qui tourne a vide. Ctrl+C existe. Ce n'est pas une honte. C'est un frein d'urgence.

### Ce que ce n'est pas

Une boucle, ce n'est pas "de la magie qui comprend ton intention". Ce n'est pas non plus une excuse pour boucler a l'infini "parce que ca marchera bien". Ce n'est pas la meme chose qu'une condition : la condition decide, la boucle repete. Et ce n'est surtout pas obligatoire d'utiliser **break** et **continue** partout. Ce sont des outils. Pas une preuve de niveau. Sam dit a ses eleves : "si tu mets **break** partout, je te demande pourquoi a voix haute". Souvent, une condition de boucle plus claire suffit.

Ce n'est pas non plus "for pour tout". Parfois **while** est le bon outil : attendre une reponse valide, un menu, un essai limite. Choisis selon l'intention, pas selon l'habitude.

`for i in range(1, 6)` : tu as un compteur qui va de 1 a 5 (le 6 est exclu). A chaque tour, tu fais quelque chose. `while vies > 0` : tant qu'il reste des vies, tu continues - mais tu dois diminuer **vies**, sinon tu tournes sans fin. Lea visualise une file d'attente : chaque personne passe une fois. Max visualise un compte a rebours. Sam visualise une liste de questions. Trois images, meme idee : repeter une action selon une regle claire.

Chez DanielCraft, on aime aussi l'image du "gardien a la porte" : tant que la condition est vraie, on laisse passer un tour. Des que la condition devient fausse, on ferme. Si tu oublies de changer la cle, le gardien ne ferme jamais.

#### ⚠ ATTENTION

Dans un **while**, change toujours la variable de condition. Sinon : boucle infinie. Ctrl+C pour couper.

## for, while, listes

```
for i in range(1, 6):
    print("Tour", i)

for i in range(5):      # 0..4
    print(i)

for i in range(0, 10, 2): # 0,2,4,6,8
    print(i)
```

```
vies = 3
while vies > 0:
    print("Il reste", vies, "vies")
    vies = vies - 1
```

Boucler sur une liste :

```
fruits = ["pomme", "banane", "kiwi"]
for fruit in fruits:
    print(fruit)

for i, fruit in enumerate(fruits, start=1):
    print(i, fruit)
```

**enumerate** donne l'index et la valeur. Lea l'adore pour afficher des listes numerotees sans bricolage. Max l'utilise pour numeroter les lignes d'un devis. Une fois vu, tu le reconnaitras partout.

## break, continue, while True

```
for n in range(1, 10):
    if n == 5:
        break      # stop complet
    if n % 2 == 0:
        continue   # saute le reste du tour
    print(n)
```

**break** = je sors. **continue** = je passe au tour suivant. Utile, mais pas magique : un message clair avant le **break** aide souvent celui qui relira ton code (toi, dans quinze jours).

Motif utile :

```
while True:
    cmd = input("Commande (q pour quitter) : ").strip().lower()
    if cmd == "q":
        break
    print("Tu as tape :", cmd)
```

Sam utilise ce motif pour des menus d'exercices. Max l'a copie pour un mini outil "calculer / quitter". Chez DanielCraft, on aime les boucles qui ont une porte de sortie claire, visible des la premiere lecture.

## Exemple : table de multiplication

```
n = 8
for i in range(1, 11):
    print(f"{n} x {i} = {n * i}")
```

Refais-la en `while`. Tu verras que les deux styles existent, et que `for` est souvent plus net quand le nombre de tours est connu. Ce n'est pas une compétition. C'est un choix de lisibilité.

### ★ A RETENIR

`for` quand le parcours est connu. `while` quand tu attends qu'une condition change. Toujours une sortie claire.

## Petite histoire

Lea devait envoyer un message personnalisé à vingt prénoms. Sans boucle, elle aurait copié vingt `print`. Avec une boucle sur une liste, elle a changé un prénom, relancé, vérifié. Dix minutes. Max a fait une boucle de décompte 10...1 puis "Decollage" pour amuser son neveu. Sam a demandé trois essais de mot de passe avec `while` + compteur. L'élève qui oubliait d'incrémenter le compteur a découvert la boucle infinie, puis Ctrl+C, puis la correction. Leçon durable : la sortie n'est pas optionnelle. Elle fait partie du design.

## Erreur classique

Oublier de modifier la variable du `while` : boucle infinie. Off-by-one avec `range` : croire que `range(1, 10)` inclut 10. Utiliser `for` quand tu as vraiment besoin d'un `while` (attente d'une réponse valide), ou l'inverse. Autre piège : `break` trop tôt et croire que le reste du programme "sait" pourquoi tu es sorti - parfois un message clair avant le `break` aide. Lea a déjà laissé un script "qui ne fait rien" parce qu'un `continue` sautait tout le travail utile. Relis le corps de boucle à voix haute.

## En vrai

Fais une boucle qui compte de 10 à 1, puis affiche "Decollage". Puis demande un mot de passe jusqu'à ce que ce soit le bon (max 3 essais) avec `while` + `break`. Teste les trois cas : bon du premier coup, bon au troisième, rate les trois. Les bords comptent autant que le chemin heureux.

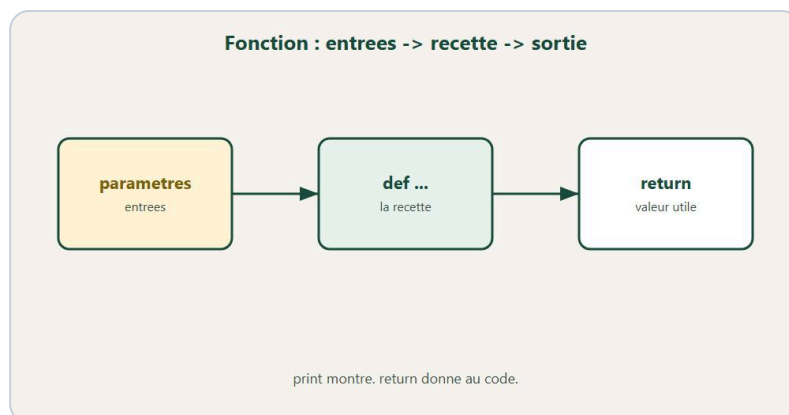
## A toi

Affiche la table de 8 (de 8x1 à 8x10) avec `for` et `range`. Puis une version `while`. Bonus : demande `n` à l'utilisateur et affiche sa table. Note en une phrase quand tu préfères `for` et quand tu préfères `while`. Garde cette phrase. Elle te servira au mini-projet.

## Zoom : range et le piège du dernier nombre

`range(1, 11)` donne 1 à 10. Le dernier nombre est exclu. C'est voulu. Ce n'est pas un bug. Si tu veux "de 1 à N inclus", pense `range(1, N + 1)`. Sam fait répéter cette phrase à ses élèves jusqu'à ce qu'elle sorte sans effort. Chez DanielCraft, on préfère une règle nette à dix essais au hasard.

## Chapitre 8 - Les fonctions



Entrees -> fonction -> return. Ecrire une fois, appeler souvent.

Une **fonction**, c'est une recette réutilisable. Tu la définis une fois. Tu l'appelles quand tu veux. Sans fonctions, tu copies le même bloc partout, tu corriges six fois la même faute, tu te perds. Avec des fonctions, tu nommes une intention : `dire_salut_a`, `moyenne`, `note_sur_20`. Lea range ses validations dans des fonctions. Max met son calcul de marge dans une fonction. Sam demande une fonction par exercice "propre".

Chez DanielCraft, on sépare tout deux gestes : `print` affiche, **`return`** renvoie une valeur au code appelant. Ce n'est pas la même chose. Beaucoup de débutants mettent des `print` partout dans la fonction et s'étonnent de ne pas pouvoir réutiliser le résultat. Apprends `return` tout. Ton code grandit mieux. Une fonction qui ne sait que "montrer" est une demo. Une fonction qui "renvoie" est une brique.

```
def dire_salut():
    print("Salut !")

dire_salut()
dire_salut()
```

### Ce que ce n'est pas

Une fonction, ce n'est pas "du code avancé obligatoire dès la ligne 1". Ce n'est pas non plus une classe (ça vient plus tard). Ce n'est pas une excuse pour faire des fonctions de 200 lignes qui font dix métiers. Une bonne fonction fait une chose claire. Et ce n'est surtout pas interchangeable avec un commentaire : le nom de la fonction doit déjà raconter l'intention. Si tu as besoin d'un roman pour expliquer `f2`, renomme.

Tu écris une fiche recette `def gateau_chocolat(nb_parts):`. Plus tard, tu appelles la recette avec 6 parts ou 8 parts. Les ingrédients changent via les **paramètres**. Le résultat peut être renvoyé (`return`) pour être utilisé ailleurs : stocke dans une variable, affiche, compare. Si la fonction ne renvoie rien d'explicite, tu obtiens `None`. Ce n'est pas un bug mystérieux. C'est le silence par défaut.

#### ★ A RETENIR

`print` montre à l'humain. `return` donne une valeur au code. Ce n'est pas interchangeable.

## Parametres, return, defaults

```
def dire_salut_a(prenom):  
    print(f"Salut {prenom}")  
  
dire_salut_a("Nora")  
dire_salut_a("Sam")
```

```
def double(n):  
    return n * 2  
  
resultat = double(4)  
print(resultat) # 8
```

Valeur par **default** :

```
def saluer(prenom="ami"):  
    print(f"Salut {prenom}")  
  
saluer()  
saluer("Lea")
```

Plusieurs parametres :

```
def moyenne(a, b):  
    return (a + b) / 2  
  
print(moyenne(12, 16))
```

Petite doc utile :

```
def carre(n):  
    # Renvoie n au carre  
    return n * n
```

Plus tard tu verras les "docstrings" (texte entre triples guillemets). Meme idee : expliquer le role. Lea ecrit une phrase. Max ecrit le "pourquoi" du coefficient. Sam exige au moins le role quand la fonction n'est pas evidente.

### \*args (aperçu avance)

Parfois tu veux un nombre variable d'arguments :

```
def somme(*nombres):  
    total = 0  
    for n in nombres:  
        total += n  
    return total  
  
print(somme(1, 2, 3, 4)) # 10
```

Tu n'es pas obligé de l'utiliser tout de suite. Mais tu le croieras dans du vrai code. Lea l'a vu dans une librairie et a compris grâce à cet aperçu. Mieux vaut l'avoir croisé une fois que de paniquer devant l'étoile.

#### ♦ ASTUCE

Ecris d'abord une petite fonction qui renvoie True/False ( `est_pair` ). Teste-la seule. Puis branche-la dans un programme plus grand.

### Pourquoi c'est bien

Tu copies moins. Le code devient plus clair. C'est plus facile à corriger, parce que tu testes une brique à la fois. Sam fait écrire `est_pair(n)` avant d'écrire un programme complet : True/False, tests avec 2, 3, 10. Si la brique est solide, le mur tient. Lea découpe ses scripts clients dès qu'elle se surprend à copier-coller un bloc. Max a appris après un devis faux : une formule à trois endroits, un oubli, un client mécontent.

### Exemple complet

```
def note_sur_20(points, total):
    if total == 0:
        return 0
    return round(points / total * 20, 1)

print(note_sur_20(15, 20))
print(note_sur_20(7, 10))
```

Regarde le garde-fou `total == 0`. Une fonction propre anticipe les cas bêtes. Chez DanielCraft, on aime ces petites protections plus que les discours sur "le clean code". Une ligne qui évite une division par zéro, c'est du soin concret.



Exemple : une recette avec entrées et résultat.

### Petite histoire

Max calculait sa marge en recopiant la formule à trois endroits. Il a changé un coefficient, oublié un endroit, envoyé un devis faux. Depuis, une seule fonction `marge(prix_vente, cout)`. Il l'appelle partout. Lea fait pareil pour formater un prix TTC. Sam note autant la clarté du nom que le résultat. Trois pratiques, une leçon : une idée, un endroit.



Autre scene DanielCraft : Lea ouvre un vieux script. Elle voit `calc`, `calc2`, `calc_final`. Elle renomme, extrait, respire. Le client n'a rien vu. Elle, si : elle peut enfin modifier sans peur.

## Erreur classique

Oublier `return` et croire que `print` "renvoie". Confondre parametre et variable globale. Mettre trop de logique dans une seule fonction. Autre piege : appeler la fonction sans parentheses dans un contexte ou tu voulais le resultat (`print(double)` affiche la fonction, pas `8`). Tu veux `print(double(4))`. Et encore : une fonction "fourre-tout" qui lit un fichier, calcule, et envoie un mail. Decoupe. Tu respireras.

## En vrai

Ecris `est_pair(n)` qui renvoie True/False. Teste avec 2, 3, 10. Puis `presenter(prenom, age)` qui retourne une phrase (pas print) : tu printes le resultat a l'exterieur. Sens la difference : la fonction donne, le programme montre. C'est le geste qui change tout pour la suite (tests, reutilisation, classes).

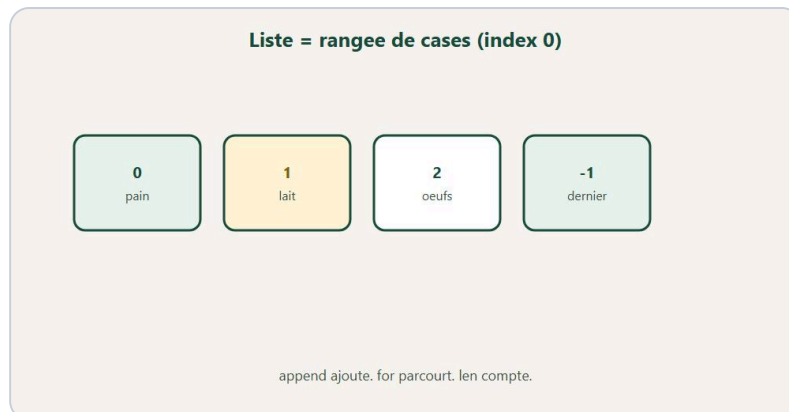
## A toi

Ecris `moyenne(a, b)` qui renvoie la moyenne. Puis `moyenne3(a, b, c)`. Teste. Bonus : gere le cas ou tu recois des textes convertibles en `float` a l'interieur de la fonction. Super bonus : ajoute un garde-fou si la conversion echoue (message clair, ou `None`).

## Zoom : nommer, c'est deja concevoir

Avant d'ecrire le corps, dis le nom a voix haute. `calculer_ttc`, `est_majeur`, `charger_notes`. Si tu galeres a nommer, c'est souvent que la fonction fait trop de choses. Chez DanielCraft, on prefere trois noms ennuyeux a un nom heroique flou. Lea renomme souvent avant de refactorer. Max a appris que `f` n'aide personne a 23h. Sam barre au tableau les noms d'une lettre hors boucles courtes. Le nom est le premier test de clarte.

## Chapitre 9 - Les listes



Une liste range plusieurs valeurs, index a 0.

Une **liste** range plusieurs valeurs, une apres l'autre. Courses, notes, prenom, scores : des suites ordonnees. L'**index** commence a 0. **-1** pointe le dernier element. Ce n'est pas un detail de puriste. C'est la source d'une tonne d'erreurs "off-by-one" chez les debutants - et chez les autres aussi, les soirs de fatigue.

Chez DanielCraft, on traite la liste comme le premier vrai conteneur du quotidien. Lea stocke des URLs a verifier. Max range des lignes de fournitures. Sam construit une liste de questions. Quand tu sais ajouter, parcourir, tester **in**, trancher et trier, tu debloques la moitie des petits scripts utiles. Trois metiers, meme muscle : une rangee de cases, des gestes clairs.

```
courses = ["pain", "lait", "oeufs"]
print(courses[0]) # pain
print(courses[-1]) # oeufs (dernier)
```

### Ce que ce n'est pas

Une liste, ce n'est pas un dictionnaire : ici l'ordre et la position comptent, pas une cle nommee. Ce n'est pas non plus un tuple : la liste se modifie, le tuple est fige. Ce n'est pas une excuse pour tout mettre dans une seule megasoupe sans structure. Et ce n'est surtout pas "trop basique pour etre pro" : les listes sont partout, y compris dans du code serieux. Lea dit : "si je dois dire le troisieme element, c'est une liste".

Une rangee de cases numerotees. La case 0 contient "pain". Tu peux remplacer, ajouter a la fin ( **append** ), inserer au debut, retirer. **len(courses)** compte. Une boucle **for item in courses** visite chaque case. Une tranche **nums[1:4]** decoupe une portion sans tout casser. Si tu imagines bien la rangee, les methodes deviennent evidentes. Chez DanielCraft, on dessine parfois ces cases au tableau. Une image nette bat dix definitions abstraites.

Max visualise ses fournitures. Sam visualise ses questions. Lea visualise une todo. Trois rangees, meme logique.

#### ⚠ ATTENTION

Le premier element est a l'index 0, pas 1. Le dernier index valide vaut **len - 1**, ou utilise **-1**.

## Modifier, ajouter, parcourir

```
courses[1] = "lait d'avoine"
print(len(courses))

courses.append("beurre")
courses.insert(0, "eau")
dernier = courses.pop()
courses.remove("pain") # enleve la premiere occurrence

for item in courses:
    print("-", item)

if "pain" in courses:
    print("On a du pain")
```

`in` se lit naturellement. Lea l'utilise pour eviter les doublons avant d'ajouter. Sam l'utilise dans des exercices "est-ce que ce mot est dans la liste ?". Max l'utilise avant d'ajouter une fourniture deja presentee.

## Tranches, tri, comprehension

```
nums = [0, 1, 2, 3, 4, 5]
print(nums[1:4]) # 1,2,3
print(nums[:3]) # 0,1,2
print(nums[3:]) # 3,4,5
print(nums[::-1]) # inverse
```

```
notes = [12, 8, 15, 10]
print(sorted(notes)) # nouvelle liste
notes.sort() # modifie sur place
notes.sort(reverse=True)
```

Attention : `sorted` renvoie une nouvelle liste, `sort` modifie sur place. Lea prefere souvent `sorted` pour ne pas ecraser l'original sans le vouloir. Chez DanielCraft, on insiste sur cette difference parce qu'elle casse des scripts "qui marchaient hier".

List comprehension (avance soft) :

```
carres = [n * n for n in range(1, 6)]
print(carres) # [1, 4, 9, 16, 25]

pairs = [n for n in range(10) if n % 2 == 0]
```

C'est une boucle `for` en une ligne. Lisible si c'est court. Sinon, garde une boucle classique. On refuse le style "incomprehension" deguisee en elegance.

## Tuple et set (aperçus)

```
point = (10, 20)
# point[0] = 11 # erreur : tuple non modifiable
```

Utile pour des paires fixes (coordonnees, resultat double...).

```
tags = {"python", "debutant", "python"}
print(tags) # les doublons sautent
```

Le `set` enleve les doublons. Max l'a utilise sans le savoir quand Sam lui a montre `set(liste)`. Aperçu suffisant pour ce livre. Tu n'as pas a devenir expert des ensembles. Tu as a reconnaitre le mot.

#### ♦ ASTUCE

Pour une moyenne : boucle + total + `len`. Verifie a la main une fois. Puis seulement, joue avec une comprehension.

## Petite histoire

Sam a fait calculer une moyenne a partir d'une liste de notes. Premier reflexe eleve : additionner a la main quatre `print`. Deuxieme reflexe : boucle + total + `len`. Troisieme : comprehension pour filtrer les notes  $\geq 10$ . Lea, elle, inverse une liste de taches avec `[::-1]` pour afficher la plus recente en premier. Max construit sa liste de jeux, ajoute un titre, affiche le dernier. Trois gestes, une meme famille d'outils. Chez DanielCraft, on celebre ce moment : tu ne manipules plus "une variable, puis une autre", tu manipules une collection.

## Erreur classique

Croire que l'index commence a 1. Modifier une liste pendant que tu la parcoures sans savoir ce que tu fais. Confondre `sorted` et `sort`. Faire `courses[len(courses)]` et sortir des bornes (le dernier index valide est `len - 1`, ou utilise `-1`). Autre piege : `remove` d'une valeur absente, qui leve une erreur - teste avec `in` avant si besoin. Lea a deja perdu vingt minutes sur un `IndexError` "inexplicable" : elle comptait comme un humain, a partir de 1.

## En vrai

Fais une liste de notes. Calcule la moyenne avec une boucle. Puis, a partir de `[1,2,3,4,5,6]`, cree la liste des pairs avec une comprehension. Verifie a la main. L'habitude de verifier bat la confiance aveugle.

## A toi

Liste de 4 jeux. Ajoute-en un avec `append`. Affiche tout avec une boucle. Affiche aussi le dernier avec `[-1]`. Bonus : trie la liste et affiche les deux premiers avec une tranche. Garde ce fichier. Au chapitre dictionnaires, tu mettras des fiches a la place de simples chaines.

## Zoom : index 0, pourquoi ?

Les ordinateurs comptent souvent depuis zero. Ce n'est pas pour t'embeter. C'est une convention utile pour les calculs d'adresse. Toi, tu retiens une regle nette : premier element = index 0, dernier = index -1. Si tu hesites, affiche `len` et teste. Chez DanielCraft, on prefere un petit test a une longue peur.

## Chapitre 10 - Les dictionnaires



Dictionnaire : etiquettes + valeurs.

Un **dictionnaire**, ce n'est pas une liste numerotee. C'est une fiche avec des **cles** : "prenom", "score", "actif". Tu ranges des paires cle -> valeur. Tu retrouves une info par son nom, pas seulement par sa position. En vrai projet, tu croises ca partout : configs, joueurs, produits, reponses d'API, fichiers JSON. Des que les donnees ont des "etiquettes", le dictionnaire devient ton ami.

Chez DanielCraft, on presente le dictionnaire juste apres la liste parce que les deux se combinent : une liste de dictionnaires, c'est deja une mini base de donnees en memoire. Lea stocke des clients (anonymises). Max stocke des produits {nom, prix}. Sam stocke des eleves. Meme structure, metiers differents. Tu vas reconnaitre ce motif dans le reste du livre, puis dans le livre Python pratique.

```
joueur = {
    "prenom": "Sam",
    "score": 120,
    "actif": True,
}

print(joueur["prenom"])
joueur["score"] = joueur["score"] + 10
joueur["niveau"] = 2 # nouvelle cle
```

### Ce que ce n'est pas

Un dictionnaire, ce n'est pas un fichier Word. Ce n'est pas non plus "obligatoirement du JSON" : **JSON** est un format texte ; le dictionnaire Python est une structure en memoire. Ils se ressemblent, ils se parlent via `json.load` / `json.dump`, mais ce n'est pas identique. Ce n'est pas une excuse pour des cles magiques eparpillees ( "n", "x2", "data" ). Et ce n'est surtout pas safe d'accéder avec `dico[cle]` si la cle peut manquer : prefere parfois `.get`.

Ce n'est pas non plus un remplacement de la liste. Si l'ordre et la position comptent plus que le nom, reste sur une liste. Si tu cherches "par etiquette", prends un dico. Lea dit : "si je dois expliquer la donnee a un client, j'utilise des cles lisibles".

Une fiche cartonnee avec des etiquettes. Tu cherches "score", tu lis 120. Tu changes le score. Tu ajoutes "niveau". Tu parcours toutes les etiquettes avec `.items()`. Si une etiquette n'existe pas, `.get("niveau", 1)` te donne une valeur par default au lieu d'exploser. La liste, elle, serait "premiere case, deuxieme case". Ici, tu nommes. Chez DanielCraft, on aime les structures qui se lisent a voix haute sans jargon.

Lea visualise une fiche client. Max visualise une fiche produit. Sam visualise une fiche eleve. Trois fiches, meme geste : ouvrir, lire une cle, modifier, sauvegarder plus tard.

#### ♦ ASTUCE

Prefere `.get("cle", default)` quand une cle peut manquer. Tu evites une `KeyError` pour une info optionnelle.

### Cle absente, parcours, liste de dicos

```
print(joueur.get("niveau", 1))
```

Si "niveau" n'existe pas, tu obtiens 1. Avec `joueur["niveau"]` direct : erreur si absent. Cette difference parait petite. Elle evite des plantages sur des donnees incompletes.

```
for cle, valeur in joueur.items():
    print(cle, "->", valeur)
```

```
print(joueur.keys())
print(joueur.values())
```

```
equipe = [
    {"prenom": "Sam", "score": 120},
    {"prenom": "Lea", "score": 95},
]
print(equipe[0]["prenom"])

for j in equipe:
    print(f"{j['prenom']} : {j['score']}")
```

Cette forme **liste de dictionnaires** est partout. Apprends a la lire a voix haute : "pour chaque joueur j dans equipe, affiche prenom et score". Sam fait repeter cette phrase. Max l'a reutilisee pour ses produits. Lea pour ses projets.

### Compter avec un dico

```
mots = ["chat", "chien", "chat", "oiseau", "chat"]
compteur = {}
for mot in mots:
    compteur[mot] = compteur.get(mot, 0) + 1
print(compteur)
```

Pattern tres courant. Sam le montre pour compter des reponses. Lea l'utilise pour des tags. Max pour des categories de depense. Une fois vu, tu le reconnaitras dans beaucoup de tutos. Tu n'as pas besoin d'une bibliotheque speciale pour ce geste debutant.

## JSON dans la tete

Un dictionnaire Python ressemble beaucoup a du JSON. Tu t'en serviras pour sauvegarder des configs, des scores, des echanges avec des services. Au chapitre fichiers, tu ecriras vraiment `joueur.json`. Ici, retiens l'idee : des nommees, valeurs typees, structure imbriquee possible. Chez DanielCraft, on prepare le terrain avant d'ouvrir le fichier : d'abord penser la structure, ensuite ecrire.

### ★ A RETENIR

Liste = position. Dictionnaire = etiquette. Liste de dicos = plusieurs fiches.

## Petite histoire

Lea recevait des infos "en vrac" dans quatre variables. Elle les a rangees dans un dico `projet`. Le script est devenu plus court, les affichages plus clairs, la sauvegarde JSON evidente. Max a fait une liste de 3 produits et calcule le total des prix avec une boucle. Sam a cree une fiche eleve, modifie la moyenne, reaffiche. Chez DanielCraft, on celebre ce moment : tu ne manipules plus "des variables perdues", tu manipules une structure. C'est le passage du bricolage a l'outil.

## Erreur classique

Typer une cle ( `"Prenom"` vs `"prenom"` ). Oublier les guillemets autour de la cle. Croire que l'ordre des cle "doit" toujours etre alphabetique pour que ca marche (ce n'est pas ta priorite debutant). Acceder sans `.get` a une cle optionnelle. Autre piege : modifier un dico en le parcourant de facon hasardeuse - construis plutot une nouvelle structure si tu filtres. Lea a deja perdu une soiree sur une `KeyError` "inexplicable" : la cle etait la, mais avec une majuscule en trop.

## En vrai

Fais une petite "fiche eleve" (nom, moyenne, classe). Modifie la moyenne. Reaffiche. Puis une liste de 3 produits `{nom, prix}` et le total des prix. Verifie a la main le total une fois. L'habitude de verifier bat la confiance aveugle.

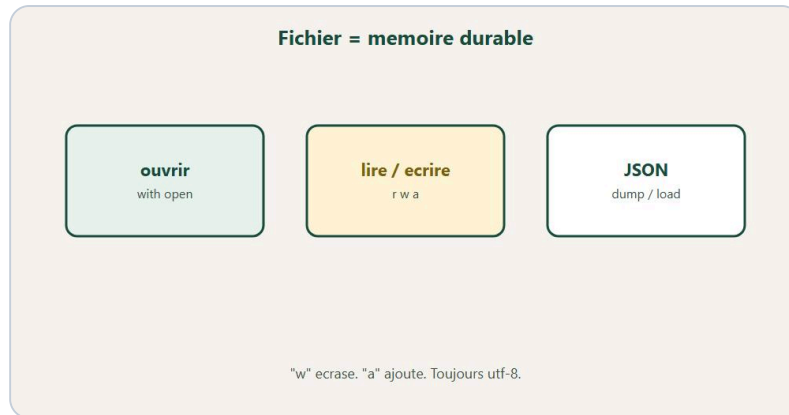
## A toi

Cree un dico `livre` avec titre, pages, lu (`True/False`). Affiche une phrase complete. Ajoute une cle `auteur`. Bonus : mets deux livres dans une liste et affiche ceux avec `lu == True`. Garde ce fichier. Au chapitre suivant, tu pourras le sauver en JSON.

## Zoom : dico vs liste, comment choisir

Si tu dis "le troisieme element", pense liste. Si tu dis "le score du joueur", pense dictionnaire. Si tu dis "tous les joueurs, et pour chacun un score", pense liste de dictionnaires. Cette phrase simple evite des heures de confusion. Chez DanielCraft, on la recolte mentalement avant chaque structure. Tu n'as pas besoin d'etre architecte. Tu as besoin d'un choix conscient.

## Chapitre 11 - Lire et ecrire un fichier



with open : lire, ecrire, sauver en JSON.

Parfois tu veux garder des infos hors du programme. Sinon tout disparaît à la fermeture. Un score, un journal, une config, une liste de tâches : le fichier est la mémoire durable du débutant. Sans fichiers, tu rejoues tout à chaque lancement. Avec fichiers, ton script commence à ressembler à un petit outil. En 2026, même un outil perso "pour moi seul" gagne à laisser une trace claire sur le disque.

Chez DanielCraft, on insiste sur `with open(...)` dès le premier exemple. `with` ferme le fichier proprement, même si une erreur survient au milieu. Lea a déjà laissé des fichiers verrouillés en oubliant de fermer. Max a écrasé un journal avec le mode `"w"` par accident. Sam fait écrire puis relire systématiquement : la boucle "écris / lis / vérifie" forme mieux qu'un discours. Tu n'as pas besoin d'une base de données pour commencer. Tu as besoin d'un fichier que tu comprends.

### Ce que ce n'est pas

Écrire un fichier, ce n'est pas "hacker le système". Ce n'est pas non plus une base de données complète. Ce n'est pas automatique : tu choisis le nom, le mode, l'encodage. Et ce n'est surtout pas anodin d'ouvrir en `"w"` : ce mode écrase sans cérémonie. Si tu voulais ajouter, c'est `"a"`. Si tu voulais lire, c'est `"r"`. Trois lettres, trois intentions. Mélange-les, et tu pleures.

Ce n'est pas non plus "seulement du texte brut". JSON te permettra de sauver des structures (dicos, listes) de façon standard. Le texte brut reste parfait pour un journal ou un export simple.

Tu ouvres un carnet (`open`). Tu écris des lignes (`write`). Tu refermes (`with` s'en charge). Plus tard, tu rouvres et tu lis (`read` ou ligne par ligne). JSON, c'est le même carnet, mais structuré comme un dictionnaire : pratique pour un score ou une config. `pathlib` est une façon plus moderne de parler des chemins. Les deux styles cohabitent. Ici, tu apprends d'abord `open` + `json`. Chez DanielCraft, on pose le geste simple avant l'outil élégant.

Lea voit un journal client. Max voit un score de jeu. Sam voit une liste de notes. Trois carnets, même geste : ouvrir, écrire, relire, vérifier.

### Écrire, lire, modes

```
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Salut\n")
    f.write("Deuxieme ligne\n")
```



```
with open("notes.txt", "r", encoding="utf-8") as f:
    contenu = f.read()
print(contenu)
```

```
with open("notes.txt", "r", encoding="utf-8") as f:
    for ligne in f:
        print(">", ligne.strip())
```

Modes utiles : `"w"` écrit (et efface). `"a"` ajoute à la fin. `"r"` lit seulement. Toujours `encoding="utf-8"` si tu touches aux accents. Lea l'a écrit par réflexe. Max l'a appris après un "Leo" transformé en bizarre. Sam le met dans la checklist du tableau.

## JSON (très utile)

```
import json

data = {"prenom": "Sam", "score": 120}

with open("joueur.json", "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False, indent=2)

with open("joueur.json", "r", encoding="utf-8") as f:
    charge = json.load(f)

print(charge["score"])
```

JSON = format texte standard pour échanger des données. Parfait pour sauvegarder un score ou une config. `ensure_ascii=False` garde les accents lisibles. `indent=2` rend le fichier humainement relisible. Chez DanielCraft, on aime les fichiers que tu peux ouvrir et comprendre sans outil spécial. Si tu ne peux pas lire ton JSON à l'œil, tu auras du mal à déboguer.

## pathlib (aperçu)

```
from pathlib import Path

p = Path("notes.txt")
print(p.exists())
print(p.read_text(encoding="utf-8"))
```

Plus moderne que `open` seul pour certains cas. Les deux cohabitent. Tu n'as pas à tout migrer aujourd'hui. Tu dois reconnaître le mot. Le livre Python pratique approfondira `pathlib`. Ici, l'aperçu suffit pour ne pas être surpris.



Exemple : sauvegarder un score dans `joueur.json`.

## Petite histoire

Lea a créé `journal.txt` en mode `"a"`. Chaque lancement ajoute une ligne datée (plus tard avec `datetime`). Elle relance deux fois, ouvre le fichier, voit s'allonger. Sensation concrète : le programme laisse une trace. Max a sauve son score en JSON, ferme le terminal, relance, recharge, sourit. Sam demande ce mini défi à toute la classe avant le chapitre modules. Ceux qui écrasent avec `"w"` apprennent vite la différence. Une erreur, une histoire, une habitude.

## Erreur classique

Mauvais dossier : "fichier introuvable". Oublier `encoding="utf-8"` et foire les accents. Mode `"w"` qui écrase sans prévenir. Chemin absolu casse quand tu changes de machine. Autre piège : croire que `json.dump` "écrit tout seul" sans `open` - non, tu fournis le fichier ouvert. Lis l'exemple encore une fois si besoin. Lea a aussi confondu `json.dump` (vers un fichier) et `json.dumps` (vers une chaîne). Une lettre de différence, deux usages.

## En vrai

Ajoute 3 lignes dans un journal `journal.txt` en mode `"a"`. Relance 2 fois. Vérifie que ça s'allonge. Puis programme qui demande un score, le sauve en JSON, le recharge, l'affiche. Fais les deux dans le même dossier. Note le chemin exact du terminal. Le "fichier introuvable" est souvent un problème de dossier courant, pas de Python.

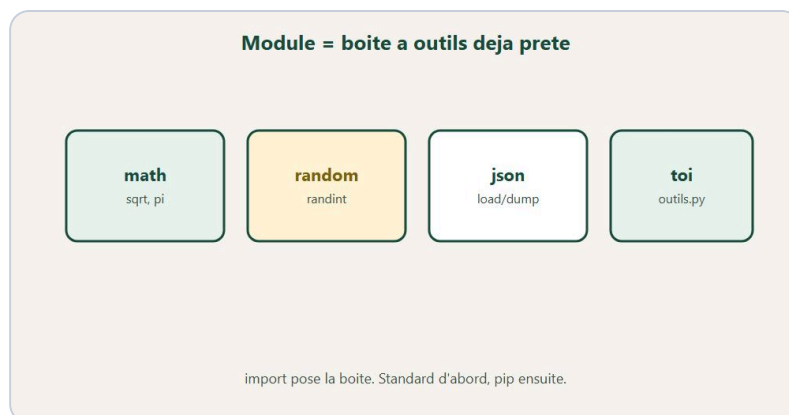
## A toi

Ecris ton prénom dans `moi.txt`. Relis le fichier et affiche-le. Puis sauvegarde un petit dico dans `moi.json` et recharge-le. Bonus : si le JSON existe déjà, affiche l'ancien score avant d'en écrire un nouveau. Garde ces fichiers. Ils serviront d'exemples dans les ateliers.

## Zoom : w, a, r en une phrase

`"r"` lit sans toucher. `"a"` ajoute à la fin. `"w"` écrase et recommence. Si tu hésites, demande-toi : "est-ce que je peux me permettre de perdre le contenu actuel ?" Si non, ce n'est probablement pas `"w"`. Chez DanielCraft, cette question avant chaque `open` évite des drames du dimanche soir.

## Chapitre 12 - Les modules (import)



import : poser une boîte à outils sur la table.

Un module, c'est une boîte à outils déjà prête. Au lieu de reinventer la racine carrée, tu `import math`. Au lieu de bricoler un tirage au sort douteux, tu `import random`. Au lieu de parser des dates à la main, tu regardes `datetime`. Python standard est riche. Apprends à chercher dedans avant d'installer le monde entier avec `pip`. En 2026, la tentation du "installe ce paquet" est partout. Resiste un peu. Souvent, la bibliothèque standard suffit pour un début solide.

Chez DanielCraft, on adore ce réflexe : module standard d'abord, bibliothèque tierce ensuite. Lea a perdu du temps à installer des paquets pour des choses que `pathlib` faisait déjà. Max a découvert `random.randint` et a fait un de en trois lignes. Sam fait tirer des conseils au hasard pour montrer qu'`import` n'est pas intimidant. Trois chemins, même leçon : tu n'es pas seul face à la machine. Des outils existent. Apprends à les appeler.

```
import math
print(math.sqrt(16))
print(math.pi)
```

### Ce que ce n'est pas

Un module, ce n'est pas un virus mystérieux. Ce n'est pas non plus "du code que tu ne peux pas comprendre". Souvent, tu peux ouvrir la doc ou même le fichier source plus tard. Ce n'est pas obligatoire d'utiliser `from math import *` (mauvaise idée : tu pollues ton espace de noms). Et ce n'est surtout pas la même chose qu'une fonction : le module contient des fonctions, des constantes, parfois des classes.

Ce n'est pas non plus "pip install pour tout". `pip` est utile. Il n'est pas le premier geste. D'abord `import` ce qui est déjà là. Ensuite seulement, une dépendance claire, isolée plus tard dans un venv (livre pratique).

Tu as ton atelier. Sur l'étagère, des boîtes étiquetées `math`, `random`, `json`, `datetime`. `import math` pose la boîte sur la table. Tu appelles `math.sqrt`. `from math import sqrt` sort juste l'outil `sqrt` et le pose devant toi. Quand ton projet grandit, tu crées ta propre boîte `outils.py` et tu l'importes. C'est le même geste. Tu passes de consommateur à auteur de module. Chez DanielCraft, ce passage est un rite de passage doux : tu ranges ton propre code comme Python range le sien.

Lea importe `json` pour ses configs. Max importe `random` pour ses jeux. Sam importe `datetime` pour dater un quiz. Puis chacun crée un petit module perso. Le geste devient naturel.

## random, datetime, from import

```
import random

print(random.randint(1, 6)) # de 1 a 6 inclus
print(random.choice(["rouge", "vert", "bleu"]))
print(random.random())      # flottant entre 0 et 1

cartes = [1, 2, 3, 4, 5]
random.shuffle(cartes)
print(cartes)
```

```
from datetime import datetime
maintenant = datetime.now()
print(maintenant.strftime("%Y-%m-%d %H:%M"))
```

Utile pour dater un log ou un score. Lea date ses journaux. Max horodate un devis exporte. Sam affiche l'heure au debut d'un quiz.

```
from math import pi, sqrt
print(pi, sqrt(9))
```

Utile quand tu utilises deux-trois symboles souvent. Evite l'etoile `*`. Prefere la liste explicite. Ton futur toi te remerciera.

## Creer ton module

Fichier `outils.py` :

```
def double(n):
    return n * 2
```

Autre fichier :

```
import outils
print(outils.double(5))
```

Pratique quand le projet grandit. Mets les deux fichiers dans le meme dossier pour debuter. Plus tard tu apprendras les packages. Ici, le geste compte : decouper. Si `import outils` echoue, regarde d'abord le dossier courant du terminal. Neuf fois sur dix, c'est ca.

## pip (aperçu)

Pour installer une bibliotheque tierce :

```
pip install requests
```

Puis tu pourras `import requests`. On ne l'utilise pas dans ce livre. Mais tu dois connaître le réflexe : module standard d'abord, `pip` ensuite, environnement virtuel encore après. Chez DanielCraft, on refuse le "installe tout ce que le tuto cite" sans comprendre pourquoi. Une dépendance, c'est une dette. Prends-la volontairement.

## Petite histoire

Max a simulé un dé à 6 faces cinq fois, additionne, affiche. Sensation de jeu immédiate. Lea a mélangé une liste de prénoms avec `shuffle` pour un ordre de passage en réunion. Sam a codé un "oracle" : cinq conseils, un tirage au hasard. Les élèves ont ri, puis ont demandé "et si je crée mon module ?". Réponse : crée `conseils.py` avec une liste et une fonction `tire()`. C'est exactement l'esprit du chapitre. Amusement d'abord, organisation ensuite, autonomie après.

## Erreur classique

Importer sans être dans le bon dossier (ton module perso introuvable). Confondre le nom du fichier et le nom importé (`outils.py` -> `import outils`). Faire `from random import random` puis s'étonner que `random.randint` ne marche plus pareil. Autre piège : réinstaller des paquets globalement en désordre - plus tard, le venv t'aidera. Pour l'instant, reste sur la bibliothèque standard. Lea a aussi nommé un fichier `random.py` et a cassé l'import du vrai module. Évite les noms qui existent déjà.

## En vrai

Mélange une liste de 5 prénoms avec `shuffle`, affiche le premier "tirage au sort". Puis petit "oracle" : tire une phrase au hasard dans une liste de 5 conseils. Relance plusieurs fois. Observe que le hasard change. C'est voulu.

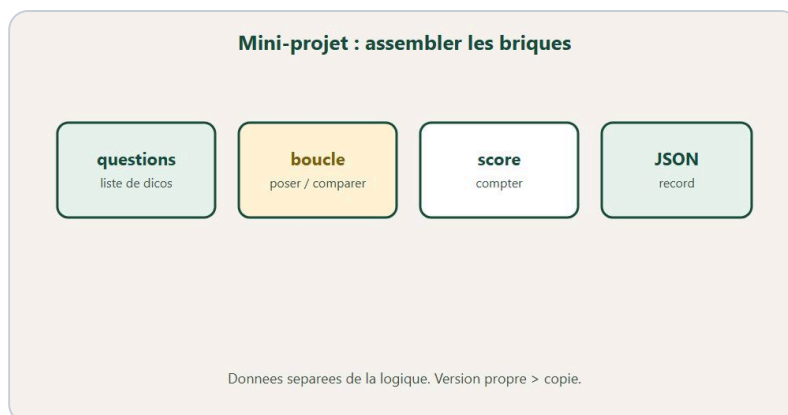
## A toi

Simule un dé à 6 faces 5 fois avec une boucle. Affiche aussi la somme des jets. Bonus : crée `outils.py` avec une fonction `moyenne(liste)` et utilise-la depuis un autre fichier. Si l'import échoue, note le dossier du terminal et corrige. Ce diagnostic te servira toute ta vie de scripteur.

## Zoom : `import math` vs `from math import sqrt`

Les deux marchent. Le premier garde le "nom de famille" (`math.sqrt`). Le second pose l'outil nu sur la table (`sqrt`). Préfère le premier quand tu débutes : tu vois d'où vient la fonction. Préfère le second quand tu utilises deux-trois symboles souvent et que le fichier reste lisible. Chez DanielCraft, on évite `import *` : trop de surprise, trop peu de clarté.

## Chapitre 13 - Mini-projet : quiz terminal



Mini-projet : questions, boucle, score, record.

On assemble ce qu'on sait : variables, conditions, boucles, listes, dictionnaires, un peu de fichiers et de modules. Ce n'est pas un examen piege. C'est une preuve pour toi : tu peux construire un petit outil complet, le lancer, le casser, le reparer. Chez DanielCraft, le mini-projet sert a ca - transformer des chapitres separees en geste fluide.

Lea a adapte ce quiz pour briefer un stagiaire. Max l'a transforme en questions metier (marges, TVA). Sam l'utilise tel quel en classe, puis demande deux questions perso. Trois versions, meme ossature. Tu n'as pas a inventer un reseau social. Tu as a finir un outil petit et net.

### Ce que ce n'est pas

Ce n'est pas "le projet final de ta carriere". Ce n'est pas non plus une excuse pour copier-coller sans comprendre. Ce n'est pas obligatoire d'aller jusqu'a la version objet des le premier jet. Version simple d'abord. Version propre ensuite. Shuffle + JSON si tu as le temps. Une marche apres l'autre. Et ce n'est surtout pas "tout faire d'un coup et ne rien finir". Un quiz de cinq questions qui marche bat un projet "parfait" abandonne a 40 %.

Tu as une liste de fiches `{question, reponse}`. Tu parcoures. Tu demandes. Tu compares avec `strip` et `lower`. Tu comptes. Tu affiches. Plus tard, tu melanges l'ordre, tu sauves le record. Le quiz n'est qu'un pretexte : la vraie competence, c'est l'assemblage. Chez DanielCraft, on repete : l'assemblage, c'est le niveau au-dessus de "je connais la syntaxe".

Lea voit un brief stagiaire. Max voit un quiz metier. Sam voit un exercice de classe. Trois pretexts, meme architecture.

### Version simple

```
score = 0

reponse = input("Capitale de la France ? ")
if reponse.strip().lower() == "paris":
    print("OK")
    score = score + 1
else:
    print("Non, c'etait Paris")

reponse = input("2 + 2 = ? ")
```

```

if reponse.strip() == "4":
    print("OK")
    score = score + 1
else:
    print("Non, 4")

reponse = input("Langage de ce livre ? ")
if reponse.strip().lower() == "python":
    print("OK")
    score = score + 1
else:
    print("Python !")

print("Score final :", score, "/ 3")

```

Ca marche. Ca se repete. D'où la version plus propre. Ne meprise pas cette version : elle prouve que tu pilotes conditions et score. Ensuite seulement, tu ranges.

## Version plus propre

```

questions = [
    {"q": "Capitale de la France ?", "a": "paris"},
    {"q": "2 + 2 = ?", "a": "4"},
    {"q": "Langage de ce livre ?", "a": "python"},
]

score = 0
for item in questions:
    rep = input(item["q"] + " ").strip().lower()
    if rep == item["a"]:
        print("OK")
        score += 1
    else:
        print("Non, reponse :", item["a"])

print(f"Score : {score} / {len(questions)}")

```

Regarde ce qui a change : les donnees sont separees de la boucle. Ajouter une question devient une ligne, pas un copier-coller de huit lignes. Lea appelle ca "arreter de se mentir sur le temps gagne". Sam appelle ca "enfin un code qu'on peut montrer".

## Version avec shuffle + JSON

```

import json
import random
from pathlib import Path

questions = [
    {"q": "Capitale de la France ?", "a": "paris"},
    {"q": "2 + 2 = ?", "a": "4"},
    {"q": "Langage de ce livre ?", "a": "python"},
    {"q": "Couleur du ciel (souvent) ?", "a": "bleu"},
]

random.shuffle(questions)
score = 0
for item in questions:
    rep = input(item["q"] + " ").strip().lower()

```

```

if rep == item["a"]:
    print("OK")
    score += 1
else:
    print("Non :", item["a"])

print(f"Score : {score}/{len(questions)}")

fichier = Path("meilleur_score.json")
meilleur = 0
if fichier.exists():
    meilleur = json.loads(fichier.read_text(encoding="utf-8")).get("score", 0)

if score > meilleur:
    fichier.write_text(json.dumps({"score": score}, indent=2), encoding="utf-8")
    print("Nouveau record !")
else:
    print("Record actuel :", meilleur)

```

## Petite histoire

Sam a fait retaper la version propre sans coller. Les élèves qui collaient bloquaient sur le JSON. Ceux qui tapaient comprenaient `Path.exists`. Max a ajouté un message selon le score (bien / moyen / à revoir) avec un `if`. Lea a mélangé les questions pour que le stagiaire ne mémorise pas l'ordre. Chez DanielCraft, on préfère ces améliorations concrètes aux réécritures cosmiques. Une feature utile, un test, un sourire.

## Erreur classique

Comparer sans `strip` / `lower`. Oublier d'initialiser `score = 0`. Ecraser le record JSON à chaque run sans lire l'ancien. Vouloir tout faire d'un coup et ne rien finir. Critères minimum : au moins 3 questions (idéalement 5), score final, comparaison souple. Autre piège : changer les questions sans relancer deux fois pour vérifier le shuffle et le record.

## En vrai

Retape la version propre sans copier-coller. Change les questions. Relance 2 fois. Puis ajoute le mélange + sauvegarde du record si tu peux. Note le temps. Tu verras que l'assemblage prend plus que la syntaxe - et c'est normal.

## A toi

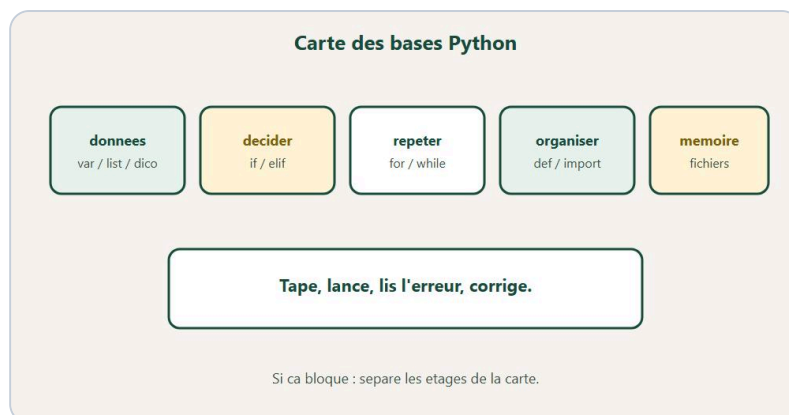
Livrable : un fichier `quiz.py` qui pose au moins 5 questions, affiche le score, et (bonus) sauve le record. Ecris aussi en trois lignes ce que tu réutiliserais dans un autre projet (boucle ? dico ? JSON ?). Range le livrable. Les ateliers suivants s'appuieront sur cette base.

## Zoom : données vs logique

Dans la version simple, données et logique sont mêlés. Dans la version propre, les questions sont des données, la boucle est de la logique. Cette séparation est le début de l'organisation. Chez DanielCraft, on la célèbre autant qu'une nouvelle fonction : demain, tu pourras charger les questions depuis un JSON sans retoucher la boucle.



## Chapitre 14 - Ce qu'il faut retenir (bases)



Carte des bases : donnees, decisions, boucles, fichiers.

Si tu ne devais garder qu'une page avant les exceptions et les classes, ce serait celle-ci. Tu as les bases : parler a l'ecran, demander, ranger, decider, repeter, decouper, sauvegarder. Ensuite on monte d'un cran. Pas pour tout oublier. Pour solidifier. Chez DanielCraft, on aime les cartes mentales honestes. Pas un glossaire de soixante mots. Une boucle de gestes que tu peux expliquer a un ami sans ouvrir le livre.

Tu n'as pas besoin de tout retenir parfaitement. Tu as besoin de reconnaitre les outils et de savoir ou les retrouver. Lea resume ca en une phrase : "je range, je decide, je repete, je sauve". Max dit : "je sais faire un petit outil". Sam dit : "je sais lire une erreur". Les trois ont raison. Choisis ta phrase. Garde-la.

### Ce que ce n'est pas

Ce chapitre n'est pas un examen surprise. Ce n'est pas non plus "la fin de Python". Ce n'est pas une liste a reciter sans comprendre. Et ce n'est surtout pas le moment de paniquer parce qu'une methode de liste t'echappe. Les gestes comptent plus que le catalogue. Si tu peux ecrire un `if`, un `for`, une fonction avec `return`, et sauver un JSON, tu es pret pour la suite.

D'abord les donnees (variables, listes, dicos). Puis les decisions (`if`). Ensuite les repetitions (boucles). Puis l'organisation (fonctions, modules). Enfin la memoire (fichiers / JSON). Quand un probleme te bloque, demande-toi ou tu es sur cette carte. Souvent, tu melanges deux etages. Separe, et ca redevient simple. Chez DanielCraft, on dessine cette carte au tableau. Tu peux la redessiner sur papier en cinq minutes.

Imagine un atelier : boites (variables), aiguillages (conditions), tapis roulant (boucles), tiroirs etiquetes (fonctions), carnets (fichiers). Tu n'as pas a tout inventer. Tu as a savoir quel meuble ouvrir.

### Les idees solides

Tu as vu `print`, `input` et les f-strings. Les variables se nomment souvent en `snake_case`. Les types courants sont `str`, `int`, `float`, `bool` et `None`. Tu sais brancher avec `if / elif / else`, et repeter avec `for / while` (plus `break` et `continue`). Les fonctions renvoient avec `return` (et tu as un aperçu de `*args`). Cote collections : listes (slices, comprehension), tuples, sets, et dictionnaires avec `.get`. Tu sais lire et ecrire des fichiers texte et du JSON, et importer des modules (`math`, `random`, `json`, `datetime` ...).

Ce n'est pas une checklist a cocher en courant. C'est un paysage. Si un mot est flou, rouvre le chapitre concerne dix minutes. Une relecture active bat une lecture passive de quarante pages.

## Habitudes qui comptent

Garde une indentation propre (4 espaces) et des noms clairs. Teste souvent. Lis les messages d'erreur sans paniquer. Prefere des petites fonctions. Pour les fichiers, prends le reflexe `with open(...)`. Retape au lieu de coller quand tu apprends. Une habitude bat dix intentions. Lea a gagne plus de temps avec des noms clairs qu'avec des astuces. Max a gagne plus de temps en testant trois valeurs qu'en "optimisant" trop tot. Sam note la methode autant que le resultat.

## Erreurs classiques a ne plus digerer

On oublie souvent `:` apres `if`, `for` ou `def`. Une mauvaise indentation fait tout planter. Comparer un nombre avec un texte (`"4"` vs `4`) donne des resultats bizarres. Un fichier "introuvable", c'est souvent le mauvais dossier. Et deux classiques : confondre `=` et `==`, ou confondre `print` et `return`. Si tu les reconnais au premier regard, tu as gagne du temps pour la vie. Chez DanielCraft, on les appelle "les pieges amis" : ils reviennent, tu les salues, tu corriges, tu avances.

## Petite histoire

Lea a ouvert son premier script trois semaines plus tard. Elle a compris encore, parce qu'elle avait nomme clair et decoupe. Max a voulu "tout refaire propre" et a perdu une soiree : mieux valait une petite amelioration. Sam a fait ecrire de memoire un `if`, un `for`, une fonction avec `return`. Ceux qui y arrivaient passaient aux exceptions sans drame. Chez DanielCraft, ce check vaut mieux qu'un discours motivationnel. Le livre ne te transforme pas. Tes gestes te transforment.

## Ce que tu peux oublier

Le nom exact de chaque methode de liste. La syntaxe parfaite de toutes les comprehensions. La peur de ne "pas etre legitime". Garde les gestes. Le reste se retrouve dans la doc ou un chapitre. Tu n'es pas un dictionnaire ambulant. Tu es un pilote qui sait ou chercher.

## Suite immediate

Deux chapitres un cran au-dessus : gerer les plantages avec `try/except`, puis modeliser avec une `class`. Puis ateliers + quiz. Tu n'es pas "fini". Tu es pret pour la suite. Respire. Puis continue.

## En vrai

Sans ouvrir le livre, ecris sur papier : une variable, une condition, une boucle, une fonction, un dico, une ligne qui ouvre un fichier. Compare ensuite. Les trous montrent ce qu'il faut relire. Pas de honte. De la carte.

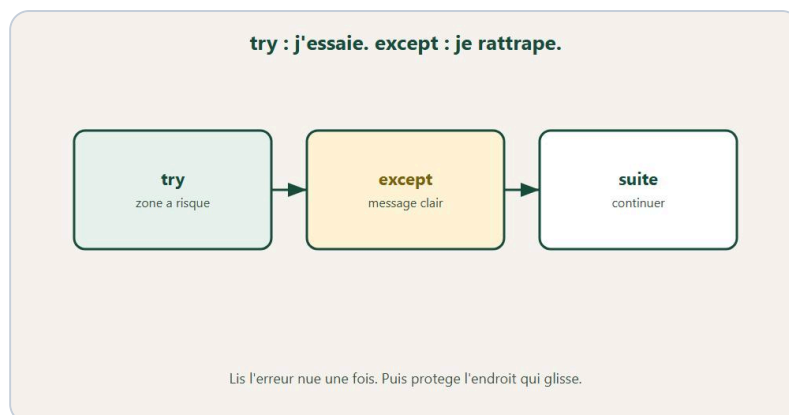
## A toi

Sans regarder : ecris de memoire un `if`, un `for`, une fonction avec `return`. Si ca vient, tu es pret. Sinon, rouvre les chapitres 6, 7 et 8 dix minutes, puis reessaie. Pas de honte. De la methode. Puis ajoute un petit JSON sauve et recharge. Si ca passe aussi, tu as le socle.

## Phrase talisman

"Tape, lance, lis l'erreur, corrige." Elle resume mieux le livre qu'une liste de mots-cles. Garde-la. Chez DanielCraft, on la recolle mentalement avant chaque session. Elle remplace la panique par un protocole.

## Chapitre 15 - Les exceptions (try / except)



try : j'essaie. except : je rattrape.

Parfois le programme plante. Pas grave. On peut rattraper. L'exemple classique : l'utilisateur tape une lettre au lieu d'un nombre. Sans `try/except`, ton script meurt avec une trace rouge. Avec `try/except`, tu affiches un message clair et tu continues - ou tu redemandes. Ce n'est pas "cacher la poussiere". C'est piloter l'imprevu.

Chez DanielCraft, on refuse deux extremes : tout laisser planter "pour apprendre", et tout avaler avec un `except:` silencieux. Lea protege les saisies clients. Max protege ses divisions. Sam force les eleves a lire d'abord l'erreur nue, puis a proteger. Comprendre avant d'envelopper. Si tu enveloppes trop tot, tu n'apprends rien. Si tu n'enveloppes jamais, ton outil est fragile des qu'un humain tape de travers.

```
texte = input("Nombre ? ")
try:
    n = int(texte)
    print("Tu as choisi", n)
except ValueError:
    print("Ce n'est pas un nombre.")
```

`try` = j'essaie. `except` = si ca casse de cette facon, je fais autre chose. Tu choisis le type d'erreur. Tu choisis le message. Tu restes responsable du comportement.

### ★ A RETENIR

Lis l'erreur nue une fois. Puis protege l'endroit qui glisse vraiment - pas tout le programme.

### Ce que ce n'est pas

`try/except`, ce n'est pas un permis de coder sale. Ce n'est pas non plus obligatoire autour de chaque ligne. Ce n'est pas interchangeable avec un `if` : parfois tu verifies avant (`if b != 0`), parfois tu attrapes apres (`ZeroDivisionError`). Et ce n'est surtout pas `except: + pass` partout. Le silence total t'empêche d'apprendre et te cache des bugs. Un filet trop large, c'est une piece sans fenetre : tu ne vois plus ou ca fuit.

Tu marches sur une planche. `try` : tu avances. Si la planche casse d'une façon prévue (`ValueError`, `FileNotFoundError` ...), `except` te donne une autre marche. `else` : "si rien n'a cassé, fais ça". `finally` : "quoi qu'il arrive, nettoie". `raise` : "je signale moi-même un problème clair". Tu ne construis pas un bunker. Tu poses des filets aux endroits où ça glisse souvent. Lea met un filet sur la saisie. Max met un filet sur la division. Sam met un filet sur l'ouverture de fichier. Trois filets, même idée.

## Plusieurs erreurs, `else`, `finally`, `raise`

```
try:
    a = int(input("a ? "))
    b = int(input("b ? "))
    print(a / b)
except ValueError:
    print("Il faut des nombres.")
except ZeroDivisionError:
    print("Division par zero interdite.")
```

```
try:
    n = int(input("Nombre ? "))
except ValueError:
    print("Nope")
else:
    print("OK, n =", n)  # seulement si pas d'erreur
finally:
    print("Toujours execute")
```

```
def age_valide(age):
    if age < 0:
        raise ValueError("Age negatif impossible")
    return age

print(age_valide(12))
# print(age_valide(-1))  # plante volontairement
```

Utile dans tes fonctions : signaler un problème clair. Lea lève des erreurs explicites plutôt que de renvoyer des valeurs magiques illégibles. Max préférerait renvoyer `-1` "pour dire erreur". Puis il a mélangé son propre `-1` comme un vrai âge. Depuis, il lève. Sam applaudit.

## Ne pas tout avaler + fichier manquant

Mauvais réflexe :

```
try:
    faire_un_truc()
except:
    pass  # silence total = tu ne vois plus rien
```

Mieux : attraper l'erreur précise, ou au moins afficher un message.

```
from pathlib import Path

p = Path("secret.txt")
try:
    print(p.read_text(encoding="utf-8"))
except FileNotFoundError:
    print("Fichier introuvable. On continue.")
```

Le message dit quoi faire à l'humain. Le type d'exception dit quoi comprendre au programmeur. Les deux comptent.

## Exemple utile : saisie solide

```
def demander_int(message):
    while True:
        try:
            return int(input(message))
        except ValueError:
            print("Un entier, s'il te plaît.")

age = demander_int("Age ? ")
print("Age =", age)
```

Tu peux réutiliser ça partout. Max l'a copié dans son devis. Sam l'exige dans le juste prix. Chez DanielCraft, une petite fonction robuste vaut mieux qu'un discours sur "la qualité". Une fois cette brique posée, tes programmes deviennent moins dramatiques : l'utilisateur se trompe, tu redemandes, la vie continue.



Exemple : on lit l'erreur, sans paniquer.

## Petite histoire

Lea a écrit un script qui plantait si le client mettait une virgule française dans un nombre. Elle a ajouté un message, puis une normalisation simple. Le client a arrêté d'appeler. Max a fait planter `int("bonjour")` volontairement, a lu `ValueError`, a compris mieux qu'avec un schéma. Sam dit : "si tu ne sais pas lire l'erreur, le `try` ne te sauvera pas - il te rendra juste aveugle plus tard".

Autre scene : Sam projette une traceback complete au tableau. Personne ne parle. Puis il souligne la derniere ligne. "Ca, c'est le GPS." Ensuite seulement, il enveloppe. Les eleves retiennent le geste, pas le jargon.

## Erreur classique

Attraper trop large. Ignorer le type d'exception. Mettre trop de logique dans le `try` (tu ne sauras plus quelle ligne a casse). Croire que `except` "repare" automatiquement - non, tu choisis quoi faire. Autre piege : masquer une vraie bug de logique derriere un filet trop large. Si ton calcul est faux, un `try` ne le rendra pas juste. Il le cachera peut-etre.

### ⚠ ATTENTION

`except:` + `pass`, c'est le silence total. Tu ne debogues plus. Tu subis.

## En vrai

Fais planter volontairement `int("bonjour")` sans `try`. Lis l'erreur jusqu'a la derniere ligne. Note le nom de l'exception. Puis protege. Ecris `diviser(a, b)` qui renvoie le resultat ou `None` si division par zero, avec un message clair. Teste avec `10, 2` puis `10, 0`. Regarde la difference de comportement. C'est ca, piloter.

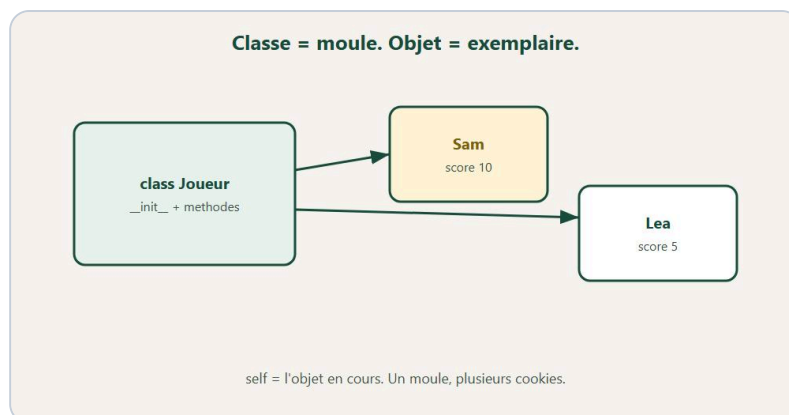
## A toi

Reprends le juste prix (ou un `int(input(...))`). Entoure la conversion avec `try/except`. Si erreur : message + redemande. Bonus : utilise `demande_int` partout ou tu convertis. Super bonus : ajoute un compteur de mauvaises saisies et un message different apres trois echecs.

## Zoom : try serre, logique claire

Garde le `try` court. Convertis. Calcule le minimum. Sors. Le message et la suite vivent dehors. Lea a longtemps mis dix lignes dans un seul `try`. Quand ca cassait, elle ne savait plus laquelle. Depuis, elle coupe. Chez DanielCraft, on dit : filet petit, idee grande. Tu verras la meme logique plus tard avec le reseau (`requests`) : timeout, status, JSON - trois filets precis, pas un mur opaque.

## Chapitre 16 - Les classes (objets)



Une classe = un moule. Un objet = un exemplaire.

Jusqu'ici : fonctions + données souvent séparées. Une classe, c'est coller les deux : "une fiche avec des actions". Tu ne ranges plus `prenom` et `score` d'un côté, `gagner` de l'autre, comme des pièces perdues. Tu dis : un `Joueur` sait son prénom, son score, et comment gagner des points. Ce n'est pas obligatoire partout. C'est puissant quand tu as plusieurs exemplaires du même genre.

Chez DanielCraft, on utilise l'image du moule à cookies. La classe = le moule. Un objet = un cookie. Lea crée des `Produit`. Max crée un `Compte`. Sam crée des `Question`. Trois moules, même idée : données + comportements ensemble. Tu modélises un bout du monde au lieu de jongler avec dix variables orphelines.

### Ce que ce n'est pas

Une classe, ce n'est pas "le niveau expert interdit aux débutants". Ce n'est pas non plus obligatoire pour trois lignes de script. Ce n'est pas une religion. Si deux fonctions et une variable suffisent, reste simple. Ce n'est pas non plus de l'héritage obligatoire dès le jour un : l'aperçu suffit. Et ce n'est surtout pas du jargon pour impressionner. `self`, c'est "moi-même, l'objet en cours". Point. Pas de mystique.

Tu fabriques un moule `Joueur`. Quand tu écris `Joueur("Sam")`, tu sors un cookie Sam avec score 0. `j.gagner(10)` demande à ce cookie d'ajouter des points. `j.présenter()` demande une phrase. Chaque objet a sa propre mémoire. Sam et Lea ne partagent pas le même score, même s'ils viennent du même moule. Si tu penses "exemplaires", tu as compris l'essentiel. Le reste, c'est de la syntaxe qui rentre en retapant.

#### ♦ ASTUCE

Commence par une classe avec deux données et une méthode. Pas dix méthodes le premier jour.

## Premiere classe

```
class Joueur:
    def __init__(self, prenom, score=0):
        self.prenom = prenom
        self.score = score

    def gagner(self, points):
        self.score += points

    def presenter(self):
        return f"{self.prenom} : {self.score} pts"

j = Joueur("Sam")
j.gagner(10)
print(j.presenter())
```

`__init__` = constructeur. Appele a la creation. `self` = l'objet en cours. Oui, c'est un peu bizarre au debut. Retape. Ca rentre. Sam fait retaper trois fois avant d'expliquer. Lea a mis deux jours a "sentir" `self`. Puis ca a clique. Normal.

## Pourquoi c'est pratique

```
equipe = [Joueur("Sam"), Joueur("Lea", 50), Joueur("Noe")]
for j in equipe:
    j.gagner(5)
    print(j.presenter())
```

Chaque joueur a son propre score. Pas besoin de `score_sam`, `score_lea` ... Une fonction ressemble a `presenter(joueur)`. Une methode ressemble a `joueur.presenter()`. Meme idee, style different. Les methodes vivent avec les donnees. Quand tu as une liste d'objets, ton code devient une petite histoire lisible.

## `__str__`, heritage, dataclass (aperçus)

```
class Joueur:
    def __init__(self, prenom, score=0):
        self.prenom = prenom
        self.score = score

    def __str__(self):
        return f"{self.prenom} ({self.score})"

print(Joueur("Sam", 12))
```

```
class Boss(Joueur):
    def gagner(self, points):
        # le boss gagne double
        self.score += points * 2
```



Une classe peut specialiser une autre. Utile plus tard. Pas obligatoire maintenant. Comprends juste : parfois un moule herite d'un autre moule et change une regle.

```
from dataclasses import dataclass

@dataclass
class Produit:
    nom: str
    prix: float

p = Produit("Stylo", 1.5)
print(p.nom, p.prix)
```

Python ecrit une partie du boilerplate pour toi. Tu le verras dans du code recent. Lea adore pour des structures simples. Sam montre les deux styles pour eviter le choc culturel plus tard. Max dit : "si c'est juste des cases a remplir, dataclass. Si j'ai des regles (retrait, depot), vraie classe."

## Quand utiliser une classe ?

Quand tu as plusieurs donnees liees et des actions dessus. Ou plusieurs "exemplaires" du meme genre (joueurs, produits, ennemis, comptes). Si c'est juste 2 fonctions et 1 variable : une fonction suffit. Pas besoin de classe partout. Chez DanielCraft, la classe est un outil, pas un trophée. La question utile : "est-ce que je modelise plusieurs exemplaires avec des regles ?" Si oui, classe. Sinon, fonctions.

## Petite histoire

Max a cree `Compte` avec `deposer` et `retirer`. Il a refuse un retrait trop grand. Sensation concrete : l'objet protege ses regles. Lea a branche une classe `Question` sur son quiz : `poser()` renvoie True/False. Sam a fait comparer deux joueurs et afficher le plus fort. Trois exercices, une meme montee en puissance : tu modelises un bout du monde.

Autre scene : Lea a voulu "tout mettre en classes" apres avoir lu un article. Son script de quinze lignes est devenu illisible. Elle a recule. Deux classes utiles, le reste en fonctions. Chez DanielCraft, on applaudit ce recul. La simplicité n'est pas un echec.

## Erreur classique

Oublier `self` dans les methodes. Appeler `Joueur.gagner(10)` au lieu de `j.gagner(10)`. Mettre toute la logique du programme dans `__init__`. Creer une classe pour une seule valeur sans comportement. Autre piege : croire que la classe "remplace" les fonctions - non, elles cohabitent. Les fonctions organisent. Les classes modelisent quand c'est utile.

### ⚠ ATTENTION

Oublier `self` casse presque toujours au debut. Relis la traceback. Ajoute `self`. Relance. C'est le rite de passage.

## En vrai

Cree 2 joueurs. Fais-les gagner des points. Affiche le plus fort. Puis classe `Question` avec `texte`, `reponse`, methode `poser()` : branche ca sur ton quiz. Sens la difference entre "deux variables" et "un objet qui sait repondre". Si ca te semble lourd pour rien, reviens aux fonctions. Si ca te semble propre, continue.

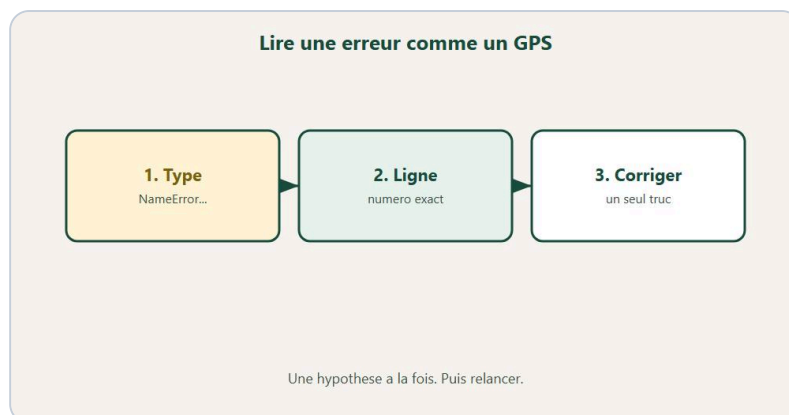
## A toi

Cree `Compte` avec `solde`. Methodes `deposer(montant)` et `retirer(montant)`. Refuse un retrait trop grand (message ou exception). Bonus : `__str__` pour afficher le solde proprement. Super bonus : une liste de comptes et un total general calcule en boucle.

## Zoom : objets et vie reelle

Un devis, c'est souvent un objet : lignes, total, TVA, statut. Un joueur de jeu, pareil. Une question de QCM, pareil. Tu n'as pas besoin d'UML. Tu as besoin de nommer la chose et ses gestes. Lea dessine parfois trois cases sur papier avant de coder. Max parle a voix haute : "le compte refuse si solde insuffisant". Sam demande aux eleves d'ecrire d'abord les methodes en francais. Puis le code suit. Chez DanielCraft, la classe commence dans la tete, pas dans le jargon.

## Chapitre 17 - Atelier : lire les erreurs



Type, ligne, corriger un seul truc, relancer.

Le terminal parle. Apprends a l'ecouter. Une erreur, c'est un GPS, pas une insulte. Chez DanielCraft, cet atelier vaut presque autant qu'un chapitre de syntaxe : si tu sais lire la derniere ligne, trouver le numero, corriger le plus petit truc, puis relancer, tu deviens autonome. Lea gagne des heures comme ca. Max a arrete d'envoyer des captures floues a Sam. Sam exige la methode avant l'aide. Trois attitudes, meme muscle : diagnostic avant bricolage.

Objectif : reconnaitre les erreurs courantes et reparer sans tout reecrire. Duree : 25 a 40 minutes. Matériel : ton quiz ou un petit script + le terminal. Pas besoin d'internet. Pas besoin d'un framework. Besoin de calme et d'une seule hypothese a la fois.

### Ce que ce n'est pas

Ce n'est pas un catalogue a memoriser par coeur. Ce n'est pas non plus "attraper toutes les erreurs avec try" avant de les comprendre. Lis d'abord. Protege ensuite. Ce n'est pas une competition de vitesse : un atelier fait a fond bat trois ateliers survoles. Et ce n'est surtout pas "demander de l'aide sans citer la derniere ligne". Chez DanielCraft, on forme des gens qui livrent un diagnostic, meme petit.

Tu es medecin du code. Le symptome, c'est le message rouge. Le type d'erreur, c'est la famille de maladie. Le numero de ligne, c'est l'endroit ou ca fait mal. Ta premiere action n'est pas d'operer au hasard. C'est de lire, localiser, changer une seule chose, retester. Lea visualise un GPS. Max visualise un devis casse qu'il relit ligne par ligne. Sam visualise un eleve qui apprend a se debrouiller avant d'appeler.

### Les familles a reconnaitre

NameError : tu utilises un nom inconnu. Souvent une faute de frappe : `scroe` au lieu de `score`.

TypeError : tu melanges mal les types, ex. `"3" + 1`. ValueError : conversion impossible, ex.

`int("bonjour")`. IndexError / KeyError : tu sors d'une liste ou tu demandes une cle absente.

```

liste = [1, 2]
print(liste[5])          # IndexError
d = {"a": 1}
print(d["b"])            # KeyError
  
```

IndentationError / SyntaxError : deux points `:` oublie, decalage foireux, parenthese non fermee.  
FileNotFoundError : mauvais chemin, mauvais dossier, fichier pas encore cree. Tu n'as pas a tout retenir.  
Tu as a reconnaitre le nom, puis agir.

## Methode DanielCraft

Lis d'abord la derniere ligne de l'erreur (le type). Va au numero de ligne. Corrige le plus petit truc, puis relance. Si besoin, entoure avec `try/except` - mais comprends d'abord. Une seule hypothese a la fois. Si tu changes dix choses, tu ne sauras plus ce qui a reparation. Note en une phrase ce que le message t'a dit. Cette note devient ton muscle.

### Exercice 1 - Syntaxe (8 min)

Casse volontairement ton quiz (enleve un `:`). Lance. Lis l'erreur. Note le type. Repare. Relance. Ecris une ligne : "ce que le message m'a dit". Si tu sautes la note, tu rates la moitie de l'atelier.

### Exercice 2 - ValueError (10 min)

Fais un `int(input(...))` et tape `abc`. Lis. Puis protege avec `try/except` comme au chapitre 15. Redemande jusqu'a obtenir un entier. Verifie aussi qu'un vrai nombre passe. Les deux chemins comptent.

### Exercice 3 - Fichier (10 min)

Ouvre un fichier qui n'existe pas. Attrape `FileNotFoundError`. Affiche un message propre. Bonus : cree le fichier s'il manque, avec un contenu par default. Lea adore ce bonus : l'outil devient accueillant au lieu d'exploser.

### Exercice 4 - Index / cle (8 min)

Provoque un IndexError et un KeyError. Corrige avec un test de longueur / `.get`. Ne te contente pas de "ca marche" : explique pourquoi ca cassait. Sam exige l'explication orale. Max se la fait a lui-meme dans le terminal.

## Petite histoire

Lea envoyait a Sam des captures floues : "ca marche pas". Sam repondait : "derniere ligne ?". Elle a appris a coller le type d'erreur. Le diagnostic est passe de vingt minutes a deux. Max, lui, changeait trois variables a la fois et ne savait plus quoi avait reparation. Depuis, une hypothese, un test. Chez DanielCraft, on repete cette scene parce qu'elle change des carrieres debutantes plus vite qu'une nouvelle syntaxe.

## Livrable

Un fichier `atelier-erreurs.md` avec : 4 types d'erreurs nommes, une capture ou copie de message, 5 lignes de lecons. Sans livrable, le cerveau classe ca comme "lu", pas comme "su". Range-le a cote de ton quiz. Tu le rouvriras.

## Erreur a eviter

Changer dix variables a la fois. Fermer le terminal sans lire. Demander de l'aide sans citer la derniere ligne de l'erreur. Croire que `try/except` remplace la comprehension. Autre piege : recopier une "solution Stack Overflow" sans comprendre le type d'erreur chez toi. Le message local reste roi.

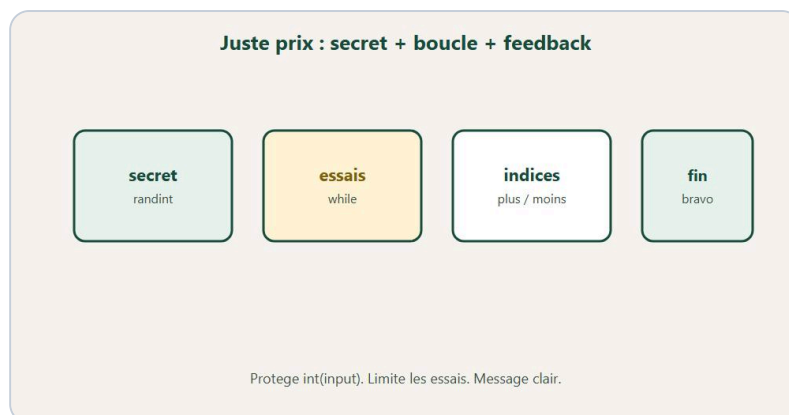
## En vrai

Prends un vieux script. Casse-le volontairement trois facons differentes. Repare. Tu entraines le muscle. C'est le vrai atelier. Si tu n'as pas de vieux script, casse volontairement le quiz du mini-projet. Meme effet.

## A toi

Fais les quatre exercices. Puis explique a voix haute, comme a Max ou Lea, la difference entre `TypeError` et `ValueError`. Si tu y arrives sans notes, c'est bon signe. Ecris ensuite le livrable. Cinq minutes d'ecriture solidifient une heure de lecture.

## Chapitre 18 - Atelier : juste prix



Juste prix : secret, boucle, indices, fin.

L'ordinateur choisit un nombre. Tu cherches. Derriere le jeu, tu revises : `random`, boucle `while`, conditions, saisie protegee, message de defaite, parfois une classe. Chez DanielCraft, on aime ces projets "jouets serieux" : tu t'amuses, tu solidifies. Lea l'a offert a un neveu, puis a ajoute un record JSON. Max a change les bornes pour faire un "juste prix devis" (trop haut / trop bas sur une estimation). Sam chronometre les essais en classe. Trois usages, meme coeur.

Objectif : implementer un juste prix solide. Duree : 35 a 50 minutes. Matériel : Python, un terminal, un fichier vide. Pas de framework. Pas d'API. Juste toi, une boucle, et un secret.

### Ce que ce n'est pas

Ce n'est pas un AAA. Ce n'est pas non plus "juste un truc enfantin". Un jeu terminal bien tenu montre que tu pilotes les boucles et les erreurs. Ce n'est pas obligatoire de tout faire en objets des le premier jet. Base d'abord. Variantes ensuite. Et ce n'est surtout pas "gagner en lisant le secret dans le code" : joue contre toi, comme si tu ne voyais pas la ligne. Sam sourit quand un eleve triche une fois, puis exige la version honnete.

Tu es face a une boite fermee. Dedans, un nombre. Chaque essai, tu frappes a la porte. "Plus grand", "plus petit", ou "bravo". Tu as un budget d'essais. Si tu depasses, la boite s'ouvre et te montre le secret - avec un message de defaite clair. Lea visualise un quiz. Max visualise une estimation de devis. Sam visualise un exercice de patience. Trois images, meme protocole : secret, boucle, feedback, sortie.

### Version de base (avec try)

```
import random

secret = random.randint(1, 20)
essais = 0
MAX = 6

while essais < MAX:
    essais += 1
    try:
        proposition = int(input(f"Essai {essais}/{MAX} - Nombre 1..20 ? "))
    except ValueError:
        print("Un nombre, merci.")
        continue
```

```

if proposition < secret:
    print("Plus grand")
elif proposition > secret:
    print("Plus petit")
else:
    print(f"Bravo ! Trouve en {essais} essais")
    break
else:
    print(f"Perdu. C'etait {secret}")

```

Le `else` du `while` s'exécute si on ne `break` pas. Pratique pour le message de défaite. Beaucoup de débutants l'ignorent. Toi, tu l'as vu. Chez DanielCraft, on souligne ce détail parce qu'il évite un `if` supplémentaire maladroit après la boucle.

## Variantes utiles

Demande si on rejoue (`o/n`). Garde le meilleur score (moins d'essais) dans un JSON. Ajoute une difficulté : facile 1-10, normal 1-20, dur 1-50. En version objet, une classe `Partie` avec `secret`, `essais` et `jouer()`. Une variante à la fois. Teste. Puis la suivante. Si tu ajoutes tout d'un coup, tu ne sauras plus ce qui casse.

## Version objet (bonus)

```

import random

class Partie:
    def __init__(self, maximum=20, max_essais=6):
        self.secret = random.randint(1, maximum)
        self.maximum = maximum
        self.max_essais = max_essais
        self.essais = 0

    def jouer(self):
        while self.essais < self.max_essais:
            self.essais += 1
            try:
                n = int(input(f"1..{self.maximum} ? "))
            except ValueError:
                print("Nombre invalide")
                continue
            if n == self.secret:
                print("Gagne")
                return True
            print("Plus grand" if n < self.secret else "Plus petit")
        print("Perdu :", self.secret)
        return False

if __name__ == "__main__":
    Partie().jouer()

```

Tu n'es pas obligé de commencer par la classe. La version simple suffit pour valider l'atelier. La classe montre que tu peux ranger le même jeu dans un objet. Utile pour la suite, pas obligatoire pour "réussir".

## Petite histoire

Sam a vu un eleve "gagner" en lisant le secret dans le code. Il a sourit, puis a dit : "maintenant cache-le vraiment et joue contre toi". Lea a ajoute le mode rejouer et a compris `while` autour du jeu entier. Max a sauve le record et a compare avec son neveu. Chez DanielCraft, le livrable compte plus que la perfection esthetique du terminal. Un jeu qui gere `abc` sans planter bat un jeu "joli" qui explose au premier typo.

## Erreur a eviter

Oublier d'incrémenter `essais`. Oublier le message de defaite. Laisser `int(input)` planter sur `abc`. Changer trop de variantes a la fois. Une variante, un test. Autre piege : mettre le secret trop visible dans les messages de debug et oublier de retirer les `print` avant de "jouer pour de vrai".

## Livrable

Un `juste_prix.py` avec au minimum : max d'essais, message de defaite, saisie protegee. Bonus : rejouer + record JSON. Bonus 2 : classe `Partie`. Ecris aussi trois lignes : ce que tu reutiliserais ailleurs (saisie solide ? boucle avec `else` ? classe ?).

## En vrai

Joue cinq parties. Note ce qui te frustre dans l'UX (messages, bornes, essais). Ameliore une seule chose. C'est comme ca qu'un jouet devient un outil. Lea a change le message "Plus grand" en "C'est plus grand" et a trouve ca plus humain. Detail. Detail qui compte.

## A toi

Implemente la base ce soir. Ajoute une variante demain. Ecris en trois lignes ce que tu reutiliserais ailleurs. Range le fichier dans ton dossier de projets. Si tu connais Git, commit. Sinon, le livre Git du parcours t'attend plus tard.



## Chapitre 19 - Atelier : organiser son code



Un role par fonction : poser, main, sauver.

Quand ca grandit, on coupe en fonctions (et parfois en classes). Pas pour faire joli. Pour retrouver ton fichier dans deux semaines sans maudire ton "toi du passe". Chez DanielCraft, l'organisation est une competence de debutant, pas un luxe de senior. Lea decoupe des qu'un script depasse une idee. Max a appris apres avoir tout mele. Sam note la clarte autant que le score du quiz. Trois chemins, meme conclusion : un nom clair et une responsabilite par fonction battent un monstre de 120 lignes.

Objectif : reprendre ton quiz et le rendre lisible. Duree : 30 a 45 minutes. Matériel : ton quiz existant (mini-projet) ou un quiz neuf de trois questions. Pas besoin d'un framework. Besoin d'un fichier que tu oses decouper.

### Ce que ce n'est pas

Ce n'est pas "creer dix fichiers pour trois lignes". Ce n'est pas non plus une architecture d'entreprise. Ce n'est pas obligatoire d'utiliser des classes si des fonctions suffisent. Une fonction / methode = une responsabilite. `poser` pose. `main` organise. Point. Et ce n'est surtout pas "renommer sans faire tourner". Chaque decoupage se valide par un lancement.

Imagine un tiroir unique ou tout est mele : questions, score, affichage, sauvegarde. Tu ouvres, tu cherches, tu pestes. Maintenant imagine trois tiroirs : `questions`, `jeu`, `main`. Tu trouves. Tu ajoutes une question sans casser le score. Lea visualise des dossiers clients. Max visualise des pieces detachees de devis. Sam visualise des fiches eleves. Chez DanielCraft, on mesure l'organisation a une question : "si tu reviens dans deux semaines, tu comprends encore ?"

### Version fonctions

```

def poser_question(texte, bonne):
    rep = input(texte + " ").strip().lower()
    if rep == bonne:
        print("OK")
        return True
    print("Non, c'etait", bonne)
    return False

def main():
  
```

```

score = 0
if poser_question("Capitale de la France ?", "paris"):
    score += 1
if poser_question("2 + 2 = ?", "4"):
    score += 1
print("Score :", score)

if __name__ == "__main__":
    main()

```

Lis a voix haute. `poser_question` pose. `main` enchaîne. Si tu ajoutes une question, tu touches surtout `main`. C'est ça, le gain.

### Pourquoi `if __name__ == "__main__" ?`

Ca veut dire : "lance `main()` seulement si on exécute ce fichier directement". Utile quand tu importeras ce fichier ailleurs. Lea l'ajoute par réflexe. Sam l'explique avec un import réel en demo. Tu n'as pas à tout comprendre du mécanisme interne aujourd'hui. Tu as à prendre l'habitude. Plus tard, quand tu importeras `poser_question` dans un test, tu seras content que le quiz ne se lance pas tout seul.

### Version classe Question

```

class Question:
    def __init__(self, texte, reponse):
        self.texte = texte
        self.reponse = reponse

    def poser(self):
        rep = input(self.texte + " ").strip().lower()
        ok = rep == self.reponse
        print("OK" if ok else f"Non : {self.reponse}")
        return ok

def main():
    quiz = [
        Question("Capitale de la France ?", "paris"),
        Question("2 + 2 = ?", "4"),
        Question("Langage de ce livre ?", "python"),
    ]
    score = sum(1 for q in quiz if q.poser())
    print(f"Score : {score}/{len(quiz)}")

if __name__ == "__main__":
    main()

```

La classe n'est pas obligatoire pour valider l'atelier. Elle montre une autre façon de ranger la même idée. Si les fonctions te suffisent, reste-y. Si tu sens que "une question" est un objet, passe à la classe.

## Decouper en fichiers (idee)

Tu peux mettre les donnees ou la classe `Question` dans `questions.py`, la logique dans `jeu.py`, et le point d'entree dans `main.py`. Meme sur un petit projet, ca clarifie. Max a resiste, puis a avoue que c'etait plus simple a montrer a son neveu : "la, les questions ; la, le jeu". Lea le fait des qu'un client peut revenir dans six mois. Sam note la clarte avant la ruse.

## Petite histoire

Lea a rouvert un script "organise" deux mois plus tard et a sourit. Elle a ajoute une question en trente secondes. L'autre script "tout en vrac" lui a coute une heure. Sam projette les deux devant la classe. Le message passe sans sermon. Chez DanielCraft, on ne vend pas l'organisation comme une morale. On la vend comme du temps retrouve. Max a compris le jour ou il a du expliquer son fichier a quelqu'un d'autre. Expliquer force a ranger.

## Erreur a eviter

Decouper trop tot (sur-abstraction). Ou trop tard (copier-coller de blocs). Renommer sans faire tourner. Melanger affichage, donnees et regles dans la meme fonction de 80 lignes. Une responsabilite, un nom clair, un test rapide. Autre piege : creer cinq fichiers vides "pour faire pro" sans y mettre de vrai decoupage. L'organisation sert le lecteur. Pas l'ego.

## Livable

Ton quiz decoupe avec au moins 2 fonctions. Bonus : classe `Question`. Bonus 2 : score en JSON. Un court `README` de 5 lignes : comment lancer. Sans `README`, ton futur toi cherchera encore "c'est quel fichier deja ?".

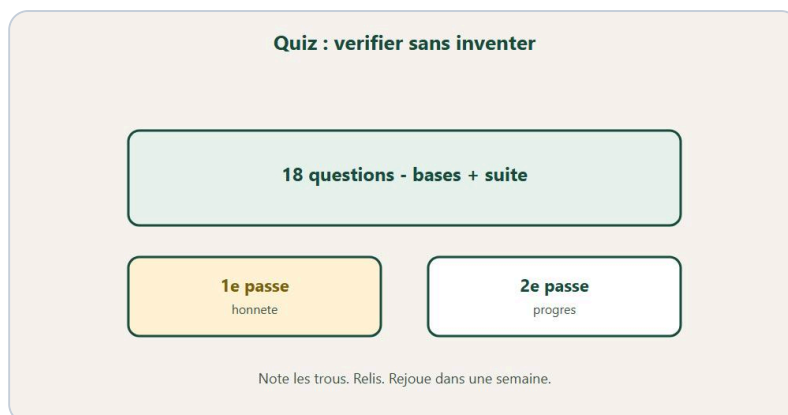
## En vrai

Rouvre le fichier demain matin sans te relire la veille. Si tu comprends en deux minutes, c'est bien organise. Sinon, renomme et decoupe encore un peu. Cette epreuve du lendemain est plus honnete qu'un sentiment de fierte le soir meme.

## A toi

Fais le decoupage ce soir. Demain, ajoute une seule amelioration (JSON, shuffle, ou fichier separe). Note le temps gagne. C'est ca, le retour sur investissement du rangement. Puis prepare-toi pour le quiz final : tu as les gestes, il reste a verifier les reflexes.

## Quiz final - Teste-toi !



Verifier ce que tu as compris, sans inventer.

Pas de piege. L'idee : verifier que tu pilotes Python avec methode - syntaxe, types, structure, fichiers, exceptions, classes. Chez DanielCraft, un quiz sert a reperer les trous, pas a humilier. Lea note ses erreurs et rouvre deux chapitres. Max refait les ateliers. Sam utilise ce QCM en fin de module. Toi, tu prends un stylo, tu coches, tu respires. Ensuite seulement, tu compares.

Reponds sans regarder. Une premiere passe honnete vaut mieux qu'une triche qui te rassure cinq minutes. Si une question te bloque, marque-la et continue. Tu reviendras. Le but n'est pas 18/18 du premier coup. Le but, c'est de savoir ou reprendre.

### ♦ ASTUCE

Fais le quiz une fois, note ton score, puis rejoue dans une semaine. Le vrai progres se voit a la deuxieme passe.

## Questions

1. 1. `print("Salut")` sert a :

- A) Lire un fichier
- B) Afficher a l'ecran
- C) Installer Python

1. 2. `input(...)` renvoie surtout :

- A) un int
- B) un str (texte)
- C) un bool

1. 3. Pour convertir `"12"` en nombre :

- A) `str(12)`
- B) `int("12")`
- C) `print("12")`

1. 4. Apres `if age >= 18:` on doit :

- A) Indenter le bloc
- B) Fermer avec `end`
- C) Mettre une virgule

1. 5. `range(1, 4)` donne :

- A) 1, 2, 3, 4
- B) 1, 2, 3
- C) 0, 1, 2, 3

1. 6. `return` dans une fonction :

- A) Affiche toujours
- B) Renvoie une valeur
- C) Efface la variable

1. 7. Dans `["a", "b"]`, index de `"a"` :

- A) 1
- B) 0
- C) 2

1. 8. Un dictionnaire associe :

- A) index numerique seulement
- B) clef -> valeurs
- C) uniquement des bool

1. 9. `with open(...) as f:` c'est bien parce que :

- A) Ca colore le texte
- B) Ca gere l'ouverture/fermeture proprement
- C) Ca remplace `print`

1. 10. `random.randint(1, 6)` :

- A) Tire un entier de 1 a 6
- B) Affiche toujours 6
- C) Cree un fichier

1. 11. `True` et `False` s'ecrivent :

- A) true / false
- B) True / False
- C) TRUE / FALSE seulement

1. 12. Indentation en Python :

- A) Optionnelle
- B) Importante (fait partie de la syntaxe)
- C) Interdite

1. 13. `try/except` sert surtout a :

- A) Accelerer le code
- B) Rattraper une erreur sans tout planter
- C) Creer une variable

1. 14. Dans une classe, `__init__` :

- A) Supprime l'objet
- B) Initialise l'objet a la creation
- C) Importe un module

1. 15. `json.dump` sert a :

- A) Lire un JSON depuis un fichier
- B) Ecrire un objet Python en JSON dans un fichier
- C) Afficher du HTML

1. 16. `liste[-1]` :

- A) Erreur toujours
- B) Dernier element
- C) Premier element

1. 17. `score += 1` equivalent a :

- A) `score = score + 1`
- B) `score = 1`
- C) `print(score)`

1. 18. `break` dans une boucle :

- A) Recommence le tour
- B) Sort de la boucle
- C) Quitte Python

## Reponses

1 B, 2 B, 3 B, 4 A, 5 B, 6 B, 7 B, 8 B, 9 B, 10 A, 11 B, 12 B, 13 B, 14 B, 15 B, 16 B, 17 A, 18 B

## Score

16-18 : nickel. 12-15 : bien, relis exceptions + classes + JSON. 8-11 : refais 5, 6, 8, 11, 15. Moins de 8 : reprends les chapitres 3 a 10 tranquillement. Puis rejoue le quiz dans une semaine. Chez DanielCraft, le vrai score, c'est la deuxieme passe.

## Petite histoire

Lea a fait 11/18 la premiere fois. Elle a rit. Puis elle a relance le juste prix, relu les exceptions, rejoue : 16/18. Max a rate la question sur `return` parce qu'il confondait encore avec `print`. Il a ecrit trois fonctions qui renvoient vraiment. Sam affiche le score moyen de la classe sans nommer personne : "on revoit indentation et dicos demain". Trois reactions, meme message : le quiz est une carte, pas un jugement.

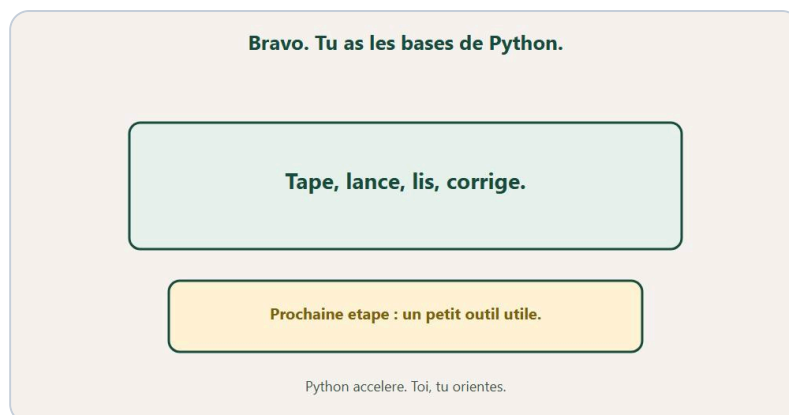
## En vrai

Coche tes mauvaises reponses. Ouvre le chapitre lie. Refais un mini exercice de cinq minutes. Pas besoin de tout relire. Un trou a la fois. Si tu as rate `try/except`, plante `int("bonjour")` volontairement. Si tu as rate les dicos, recree un mini carnet contacts. Le geste ancre mieux qu'une relecture passive.

## A toi

Ecris tes trois questions ratees (ou les plus floues). A cote, une phrase "ce que je confonds". Garde ce papier. Dans sept jours, rejoue seulement ces trois. Chez DanielCraft, on prefere trois trous bouches a un 18/18 oublie le lendemain.

## Bravo.



Bravo. Tu as les bases de Python.

Tu as fini ce livre Python. Pas "je connais tout". Mais "je sais ecrire un programme, le faire tourner, le corriger, et le structurer". C'est deja enorme. Chez DanielCraft, on considere que c'est ca, la vraie competence debutant solide en 2026. Pas reciter vingt frameworks. Piloter un fichier `.py` avec methode.

Lea repart avec des scripts utiles et des noms clairs. Max repart avec un devis calcule et un juste prix pour s'entrainer. Sam repart avec des ateliers qu'il peut rejouer en classe. Toi, tu repars avec une habitude : tape, lance, lis l'erreur, corrige. Cette habitude vaut plus qu'une certification colle au frigo.

### Ce que ce n'est pas

Ce bravo n'est pas un diplome magique. Ce n'est pas non plus "tu n'as plus rien a apprendre". Ce n'est pas une invitation a abandonner la verification humaine. Et ce n'est surtout pas le moment de tout jeter pour "refaire propre depuis zero". Ameliore. Versionne. Continue. Le perfectionnisme du grand soir casse plus de progressions que les erreurs de syntaxe.

Tu as un petit atelier. Sur l'etabli : variables, conditions, boucles, fonctions, listes, dicos, fichiers, modules, exceptions, classes. Tu sais ouvrir chaque tiroir. Tu ne connais pas encore toute la piece d'a cote (venv, API, tests). Ce n'est pas grave. Tu as les mains. Lea visualise ses exports. Max visualise ses calculs. Sam visualise ses eleves qui osent lancer un script sans paniquer. Trois images, meme bravo.

### Ce que tu sais faire

Tu sais installer et lancer Python. Tu manipules variables, types, conditions et boucles. Tu ecris des fonctions (parametres, return, apercu `*args`). Tu te debrouilles avec listes (slices, comprehensions) et dicos. Tu lis et ecris des fichiers texte et du JSON. Tu utilises des modules (`random`, `math`, `json`...). Tu geres les exceptions avec `try/except`. Tu as cree une premiere `class`. Et tu as fait un quiz et un juste prix. Relis cette liste a voix haute. C'est ton paysage, pas une liste de courses.

### Mission finale

Ameliore ton juste prix en mode "produit mini". Limite a 6 essais avec un message de defaite. Protege la saisie. Propose de rejouer. Sauve le record en JSON. En bonus, une classe `Partie`. Puis range le projet dans un dossier propre. Si tu connais deja Git, versionne-le. Sinon, le livre Git du meme parcours t'attend. Une mission petite et finie bat dix intentions vagues.



## Petite histoire

Lea a repris son premier `salut.py` trois mois plus tard. Elle a rit. Puis elle a vu le chemin : du print au JSON, des erreurs lues sans panique, un quiz range. Max a montre son devis calcule a un client sans dire "c'est du Python". Le client a vu un outil clair. Sam a vu un eleve corriger seul une IndentationError. Chez DanielCraft, ces scenes valent mieux qu'un score de quiz. Elles disent : tu pilotes.

## La suite (quand tu voudras)

Tu pourras explorer virtualenv et `pip`, puis Requests pour parler a une API. Un peu de data (pandas) ou de web (Flask/FastAPI). Des tests automatiques avec `pytest`. Et lire du code des autres sur GitHub. Le livre Python pratique du parcours continue exactement sur cette route : CSV, CLI, venv, HTTP, logs. Mais la, respire. Une competence posee bat dix tutos binge.

## Engagement de 30 jours

Chaque jour ouvré : dix minutes de Python bas risque (exo, mini script, relecture d'erreur). Chaque semaine : ameliorer un petit outil. J+30 : refaire le quiz et relancer ton juste prix. Tu transformeras ce livre en muscle. C'est le vrai bravo. Sans engagement, le livre redevient un souvenir. Avec engagement, il devient un reflexe.

## Erreur classique apres un livre

Croire que "j'ai fini" egal "je n'ai plus besoin de pratiquer". Ou vouloir enchaîner trois frameworks le lendemain. Ou tout jeter parce qu'un chapitre te semble encore flou. Autre piege : ne plus ouvrir le terminal pendant un mois, puis conclure que "Python c'est pas pour moi". Dix minutes regulieres battent une binge oubliee.

## En vrai

Ouvre ton juste prix ou ton quiz aujourd'hui. Change une seule chose utile. Lance. Souris. Range. C'est le premier jour de l'apres-livre. Chez DanielCraft, on prefere ce geste a un long discours de motivation.

## A toi

Ecris en trois phrases : (1) ce que tu sais faire maintenant, (2) le petit outil que tu vas ameliorer cette semaine, (3) ce que tu ne feras pas encore sans filet. Garde ce papier. Dans trente jours, relis-le. Tu verras le chemin.

## Un dernier rappel

Python accelere. Toi, tu orientes. Garde le gouvernail. Quand un message d'erreur apparait, souviens-toi : ce n'est pas une insulte. C'est un GPS. Lis la derniere ligne. Corrige le plus petit truc. Relance.

## Petite scene DanielCraft

Lea ouvre son premier `salut.py` et son dernier quiz le meme soir. Elle mesure le chemin sans se comparer a personne. Max lance son devis, change un taux, relance, sourit. Sam envoie a ses eleves : "vous savez lire une erreur - c'est deja un metier". Trois gestes, un meme bravo discret. Chez DanielCraft, on prefere ce silence utile aux grands discours de fin de formation.

Encore bravo. Tu parles a l'ordinateur. Et maintenant, tu geres aussi quand il rale.

Tu as les briques. Maintenant, construis.